
Machine Learning for the Detection of Atmospheric Blocking Events during European Summer

Bachelor thesis
by

Tobias Leibrock

Institute of Theoretical Informatics

Reviewer:	TT-Prof. Dr. Peer Nowack
Second Reviewer:	Jun.-Prof. Dr. Jan Stühmer
Co-Supervisor:	Dr. Markus Götz

Begin:	11.12.2023
Submission:	30.03.2024

Declaration of Authorship

I hereby declare that this document has been composed by myself and describes my own work, unless otherwise acknowledged in the text. I also declare that I have read and observed the *Satzung zur Sicherung guter wissenschaftlicher Praxis am Karlsruher Institut für Technologie*.

Karlsruhe, March 30, 2024

A handwritten signature in black ink, reading "T. Leibrode". The signature is written in a cursive style with a large, stylized 'L' at the end.

Your name

Contents

Abstract	i
Zusammenfassung	ii
1 Introduction	1
1.1 Motivation & Aim of the Study	1
1.2 Research Questions and Hypotheses	2
2 Background	4
2.1 Atmospheric Blocking & Detection Methods	4
2.2 Basics of Machine Learning	6
2.2.1 Unsupervised and Supervised Learning	8
2.2.2 Linking Atmospheric Blocking and Machine Learning	11
2.3 Related Work	11
3 Data and Methodology	13
3.1 Used Datasets	13
3.2 Tools and Libraries	15
3.3 Data Preprocessing Steps	16
3.4 Machine Learning Pipeline	19
3.4.1 Model Choice	19
3.4.2 Hyperparameter Optimization	20
3.4.3 Evaluation Metrics and Validation Methods	21
4 Implementation	25
4.1 Data Pipeline	25
4.2 Random Forest (RF)	26
4.3 Convolutional Neural Network (CNN)	27
5 Results	35
5.1 F1 Performance	35
5.2 Transferability	40
5.3 Combined Training & Model Fine-Tuning	42
6 Discussion and Conclusion	45
6.1 Method Comparison	45
6.2 Lessons Learned	47
6.3 Study Limitations	49
6.4 Conclusion and Outlook	51
Appendix	54
Bibliography	56

Abstract

Atmospheric blocking is a mid-latitude phenomenon of high-pressure regions that resist the normal westerly jet stream flow and remain in place for several days to weeks. Extreme weather events such as heat waves and cold snaps are heavily influenced by atmospheric blocking, as the persistent high-pressure region locks in extreme weather conditions. These events result in massive humanitarian and economic damage. Blocking events are notoriously difficult to detect accurately. In the past, many so-called blocking indices that depend on specific meteorological variables and thresholds have been proposed. They are often regional, seasonal, and specific to blocking types, resulting in bad performance. Recent unsupervised learning approaches to blocking detection demonstrate the potential of machine learning in blocking detection, improving performance compared to traditional indices during European summer. This thesis introduces two new supervised learning approaches using random forest classifiers (RF) and convolutional neural networks (CNN) that outperform traditional detection methods consistently for European summer atmospheric blocking detection. The final RF and CNN methods, named RF-BI and CONV-BI, show good performance on observational reanalysis data (using the ERA5 dataset) and Earth system model data (using the UKESM dataset), and outperform traditional index-based and unsupervised learning methods. In addition to geopotential height at 500 hPa (Z_{500}), RF-BI and CONV-BI are also tested on mean sea level pressure ($MSLP$) data. Both methods outperform the unsupervised method SOM-BI with ($MSLP$) data on both datasets. The results suggest a good potential for long-term trend analysis. For transferability between datasets, RF-BI and CONV-BI are trained on observational data and tested on earth system model data and vice versa. Both methods transfer learning between datasets and maintain high performance, demonstrating the potential for applications beyond the training dataset. The potential to scale with additional training data is demonstrated by training on a combination of observational and earth system model data. Overall, this thesis demonstrates the potential of supervised learning for atmospheric blocking detection by introducing two novel detection methods, RF-BI and CONV-BI, which consistently outperform traditional index-based and unsupervised methods.

Zusammenfassung

Atmosphärisches Blocking ist ein Phänomen in mittleren Breitengraden, welches durch Hochdruckgebiete gekennzeichnet ist, die dem normalen westlichen Jetstreams widerstehen und für mehrere Tage bis Wochen stationär in der Atmosphäre bleiben. Extreme Wetterereignisse wie Hitzewellen und Kälteeinbrüche werden stark durch atmosphärisches Blocking beeinflusst, da die Hochdruckregion extreme Wetterbedingungen in den betroffenen Regionen verstärkt, was zu massive humanitäre und wirtschaftliche Schäden führt. Die genaue Erkennung von Blocking ist bekanntermaßen schwierig. Bisher werden verschiedenste Blocking-Indizes verwendet, die von spezifischen meteorologischen Variablen und Schwellenwerten abhängen. Diese sind oft regional, saisonal und auf bestimmte Blocking-Typen ausgerichtet, was zu schlechter Leistung im Allgemeinen führt. Neue Ansätze benutzen unüberwachten Lernens zur Blocking-Erkennung und demonstrieren das Potenzial von maschinellem Lernen. Die Forscher erreichen auf Europäischen Sommer Blocking Ereignissen eine höhere Genauigkeit als bisherige Methoden. Diese Thesis stellt zwei neue überwachte Lernansätze vor, die Random-Forest-Klassifikatoren (RF) und Convolutional Neural Networks (CNN) nutzen und traditionelle Erkennungsmethoden für die Erkennung von atmosphärischem Blocking im europäischen Sommer konsistent übertreffen. Die endgültigen RF- und CNN-Methoden, RF-BI und CONV-BI, demonstrieren gute Leistung bei Reanalyse-Daten (ERA5-Datensatz) und Daten von Klimamodellen (UKESM-Datensatz) und übertreffen traditionelle indexbasierte und unüberwachte Lernmethoden deutlich. Zusätzlich zum Geopotential auf 500 hPa (Z_{500}) wurden RF-BI und CONV-BI auch auf Daten des mittleren Luftdrucks auf Meereshöhe ($MSLP$) getestet. Beide Methoden übertreffen die unüberwachte Methode SOM-BI mit ($MSLP$)-Daten auf beiden Datensätzen. Die Ergebnisse zeigen ein gutes Potenzial für langfristige Trendanalysen mit CONV-BI und RF-BI. Bei der Übertragbarkeit zwischen Datensätzen werden RF-BI und CONV-BI mit Beobachtungsdaten trainiert und an Daten von Klimamodellen getestet und umgekehrt. Beide Methoden übertragen die gelernten Muster zwischen Datensätzen und erreichen eine hohe Leistung auf beiden Datensätzen, was das Potenzial für Anwendungen über das Trainingsdatenset hinaus demonstriert. Skalierbarkeit mit zusätzlichen Trainingsdaten wird durch das Training aus einer Kombination von Beobachtungs- und Klimamodellen gezeigt. Insgesamt demonstriert diese Arbeit das Potenzial des überwachten Lernens für die Erkennung von atmosphärischem Blocking, indem sie zwei neuartige Erkennungsmethoden, RF-BI und CONV-BI, vorstellt, die konsistent traditionelle indexbasierte und unüberwachte Methoden übertreffen.

1. Introduction

1.1 Motivation & Aim of the Study

Climate change and global warming increase the frequency of extreme weather events around the world. Heat waves and cold spells in the European region are no exception to this global trend. These extreme weather events throughout Europe cause massive ecological and humanitarian damage, especially increased mortality among the population [1, 2]. Heat waves in the last 20 years resulted in more than 40,000 deaths across Europe in 2003 [2], and over 2,065 deaths in France during the 2006 heat wave alone [1]. Preemptive measures implemented in multiple countries such as France were able to reduce the effect of heat waves and the resulting death toll on the population greatly. Model predictions during the heat wave in 2006 estimated the death toll at 6452 with the actual number at 2065 showing a big reduction most likely related to the establishment of the National Heat Plan before the 2006 heat wave [1].

Preparation plans for heat waves have been proven to work and reduce economic and humanitarian damage from extreme weather events, including heat waves. But all of them are dependent on early warning systems and climate models that can reliably forecast future developments. The ability to increase reliable extreme weather forecasting capabilities is crucial for nations worldwide to prepare and act accordingly [2]. One of the main indicators for extreme weather events is atmospheric blocking. Blocking events are high-impact weather events in the mid-latitude atmosphere that block the usual westerly flow. The resulting condition with a high-pressure region stuck in the same geographic region for multiple days up to multiple weeks amplifies extreme conditions. This can lead to hotter heat waves, longer droughts, and colder cold spells. A good understanding of blocking events and reliable and accurate detection in earth system models and forecasts can greatly increase the detection accuracy of future extreme weather events, meanwhile reducing humanitarian and economic damage.

Aim of the Study

This thesis evaluates the possibility of using supervised machine learning approaches for the detection of blocking events in the atmosphere. Current state-of-the-art approaches to detect blocking in simulations and forecasts use multiple index-based approaches simultaneously, reducing the issues with single index-based methods. They often detect blocking events without using all available information and suffer from poor accuracy on different blocking types. With the underlying physics behind blocking formation and duration not fully understood [3] unsupervised learning approaches have been successfully used to detect blocking events without relying on specific indices or physical properties [4]. During the evaluation of unsupervised approaches, a new ground truth dataset (GTD) based on 41 years of labeled blocking data was created by experts.

“The GTD has been derived by studying every successive 5 d period from 28 May 1979 to 4 September 2019 and manually identifying whether or not a blocking high persisted across any such 5 d period [4].”

Using the GTD, this thesis aims to evaluate the potential for supervised machine learning algorithms to improve accuracy of blocking detection methods. Similarly to a previous paper comparing unsupervised learning methods to traditional index-based method by Thomas et al. [4] two meteorologicla variables including geopotential height (Z_{500}) and mean sea level pressure ($MSLP$) are used to train supervised algorithms. Explaining both variables in depth is beyond this paper, a short introduction can be found in Section 2.1 and a more detailed description is available from the European Centre for Medium-Range Weather Forecasts [5]. The accuracy of algorithms is compared on different datasets to evaluate the ability to transfer learnings between observational and earth system model data. To compare results, this study uses traditional index-based methods, unsupervised methods, and the newly introduced supervised methods. This includes a random forest classifier and a convolutional neural network model. The findings are presented with a discussion about supervised method advantages and disadvantages compared to different detection methods. In the end, a conclusion for supervised learning in the detection of blocking events is drawn with an outlook into improvements, shortcomings and future developments.

1.2 Research Questions and Hypotheses

This study aims at answering specific areas of research for the potential of supervised learning in the detection of blocking events. The following paragraphs introduce three research questions, that are focussed on this evaluation, including basic performance requirements, comparisons, algorithm setup, model configuration and data usage. When presenting the results (Section 5), this paper answers all research questions with achieved results during model training and comparison.

One of the most important research questions compares the performance of supervised methods for atmospheric blocking detection to traditional index-based and unsupervised methods. To prove the potential of supervised learning, the introduced detection methods need to match or exceed previous highest-performing detection methods.

RQ1: How does the performance of the supervised learning approach using a convolutional neural network (CNN) compare to existing unsupervised methods and index-based methods, in detecting atmospheric blocking events during European summer?

When using convolutional neural networks and random forest classifiers, hyperparameters greatly affect model performance. This is especially relevant for convolutional neural networks, because they are more complicated and vary in architectural design and configuration. The performance implications of certain hyperparameters and architectural design decisions are important to guide future work and models in the right direction.

RQ2: How do different configurations of the CNN model, such as varying architectural designs and hyperparameter choices, impact the accuracy and efficiency of blocking detection?

Supervised model training is influenced not only by hyperparameters and model configurations, but also by the underlying dataset. In the case of atmospheric blocking detection, both observational and model data are available. For many applications, it is important to consider the transferability of learning from one dataset to another. Transferability is particularly relevant when comparing two datasets from different sources, as it can demonstrate a complete understanding of blocking events by the detection methods. Training on all available data, including observational and model data, can further improve performance.

RQ3: How does the CNN model perform on simulated data when trained on observational data or a combination of observational and simulated data, and what implications does this have for the model’s applicability in real-time or near-real-time atmospheric blocking prediction?

A combination of observational and model data can enhance performance even further, increasing the available dataset size. This can demonstrate the potential to scale with data size and increase performance further, by collectin more labeled data.

2. Background

This section introduces the fundamentals of atmospheric blocking and machine learning, with a focus on concepts used throughout the thesis. It begins with an introduction focused on the detection of atmospheric blocking and the meteorological variables involved. Following this, a brief historical recap of machine learning is provided, along with the necessary basics required for this thesis. This section covers unsupervised and supervised learning, with a detailed explanation of classification tasks, random forest classifiers, and convolutional neural networks. Both random forests and convolutional neural networks are extensively utilized in this thesis.

2.1 Atmospheric Blocking & Detection Methods

By definition, atmospheric blocking events consist of high-pressure regions, blocking the usual jet stream for at least five days with extreme cases lasting multiple weeks at a time [6]. Blocking events occur in the mid-latitude region where the jet stream leads to average zonal winds blowing from west to east in the Northern hemisphere (Figure 2.1). In meteorology, these winds are referred to as 'westerlies' [7].

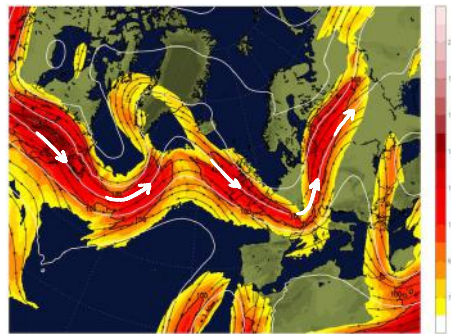


Figure 2.1: Polar jet stream over parts of America, the Atlantic, and Europe. Flow from east to west highlighted with white arrows.

As their name implies, blocking events are stationary high-pressure weather systems that temporarily and regionally obstruct the westerly flow, resulting in blocked regions of the atmosphere. This, in turn, leads to abnormal, often extreme, weather conditions within and around these high-pressure systems. In particular, prolonged blocking events are associated with heatwaves and droughts during summer and cold spells during winter [6, 7]. Experts agree on a sharp flow pattern transition from zonal to meridional flow during atmospheric blocking events on the edge of blocked regions [8]. These blocking events can be further assigned to special types with different mechanisms during onset, duration, and decay.

- **Omega blocks** special kinds of blocking events, named the Greek letter Omega (Ω), these atmospheric blocking patterns occur when a high-pressure system is flanked by two low-pressure systems (Figure 2.2.a) forming an Omega shape. The high-pressure

system forces the jet stream to flow in a wave-like pattern, creating blocking conditions in the region [6].

- **Dipole blocks** also called Rex blocks consist of a high-pressure anticyclone poleward of a low-pressure cyclone creating a blocking region with the reverse of the usual atmospheric flow (Figure 2.2.b). This anticyclone and cyclone situation is closely linked with Rossby wave breaking. Rossby waves are large-scale atmospheric waves moving in the upper atmosphere affecting weather patterns underneath by affecting jet streams and temperature distributions. Breaking of Rossby waves can lead to reversing the usual meridional flow of the atmosphere [6].

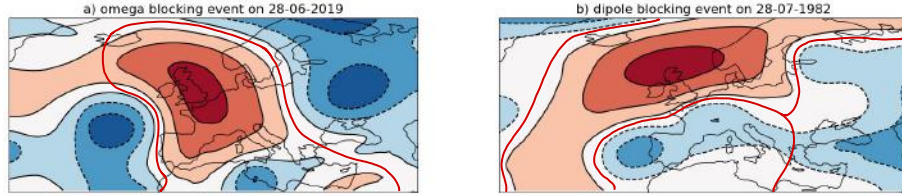


Figure 2.2: Visualization for omega atmospheric blocking type (left) over northern Europe during June 2019 and a dipole blocking event (right) over Europe during July 1982. For the dipole blocking event the high-pressure region is situated northward with a low-pressure region directly below. The Omega block is flanked by two low-pressure regions surrounding the high-pressure blocked region in the center.

When blocking events form, the westerly flow in the atmosphere is obstructed by a persisting high-pressure region. Contrary to the usual winds in parts of a blocked region, easterly winds can be observed [6]. There are multiple contributing mechanisms that have been hypothesized to lead to the occurrence of blocking, for which a detailed discussion is beyond the scope of this thesis but can be found in [6, 7]. In short, some ideas linked to the formation of blocking include: The breaking of so-called Rossby waves, which transport momentum and energy on a global scale, or strong cyclonic activity in the region upstream of the formation of blocking events. In addition, stationary atmospheric waves (called quasi-stationary Rossby waves) originating in tropical regions and moving poleward in the Northern Hemisphere deform the atmospheric contours and contribute to the onset of blocking. Similarly, regions of weak and divergent flow reduce the travel capacity of waves in the atmosphere, leading to an increased probability of blocking onset [6, 7, 9, 10].

Current Approaches to Blocking Detection

Due to the different kinds of blocking events, there is also a wide range of different detection definitions. Specific criteria are defined and agreed upon in all popular blocking methods. This includes a time range minimum of four to five days and minimal to no westerly movement of the extreme event with no increase in the surface area of the affected region. All methods work on gridded data with various meteorological variables such as geopotential height and mean sea level pressure [7].

- **Geopotential Height (Z_{500})** is a gridded meteorological variable representing the work required to lift a unit mass to a predefined height. It is also the potential energy of mass on the surface relative to mean sea level. Geopotential height is a highly dynamic variable and the resulting grid is constantly changing [5]. For blocking detection, geopotential height is one of the best variables due to the low noise level at certain pressure levels, such as 500hPa, which is commonly used [4].
- **Mean sea level pressure ($MSLP$)** is a gridded variable corresponding the pressure of the atmosphere at sea level. This corresponds to the weight of all the air inside

a column above the measured point on the surface. For land area, atmospheric pressure is adjusted to represent the sea level [5]. Mean sea level pressure is stable for long-term climate trend analysis and is stable regarding CO_2 concentrations. It is also noisier compared to geopotential height due to the effect of local weather and temperature variations [4].

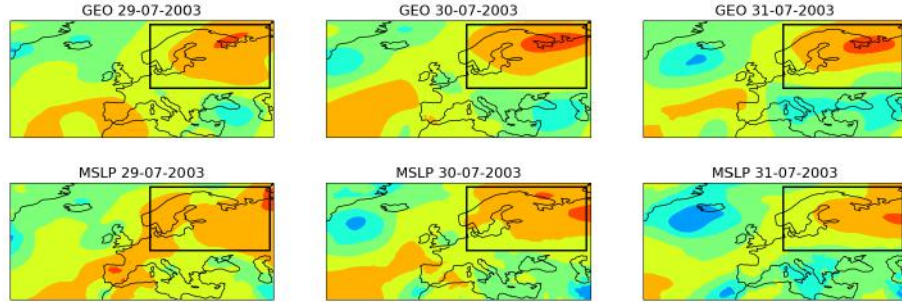


Figure 2.3: Comparison between geopotential height and mean sea level pressure during the onset of the July 2003 European heat wave. Onset of the blocking event is visible in the geopotential height data (top) compared to a noisy representation in the mean sea level pressure data (bottom). The mean sea level data representation is unable to indicate the high-pressure region due to possible noise introduced by local weather effects and temperature differences. The high-pressure region is consistent between multiple days for geopotential height.

After applying the method to the data sample, the output of the analysis is the final blocking index [6]. A subsection of indices focused on a 1D meridional gradient criteria in a predefined latitudinal band of the jet stream [6]. Some indices use a 2D approach to better capture blocking in a wide range of latitudes and to avoid relying on a specific input latitude. For this approach, Rossby waves interacting with each other and the resulting wave breaking are generally used to distinguish between normal atmospheric flow and the formation of a block [7]. Recent studies have focused on a combination of several indices to reduce the discrepancy between results and increase the reliability of a detection method [6].

Detection Issues

Reliably and accurately detecting blocking has historically been a difficult problem because different blocking indices with different strengths and weaknesses, resulting in single indices performing well for certain blocking types or regions and worse for others. Recent studies propose an array approach to combine the strengths of multiple indices, increasing accuracy compared to the indices by themselves [6]. Predicting blocking directly via the underlying physical system is hard as the understanding of the dynamics of blocking is still an open issue and it is unknown which physical systems are responsible and how they interact with each other [6, 7]. Recent work by Thomas et al. [4] proposed a different approach based on self organizing maps, an unsupervised machine learning approach, recognizing blocking patterns in the underlying data. This approach performs better than traditional blocking indices and has additional advantages such as transferability between multiple datasets [4].

2.2 Basics of Machine Learning

This section introduces the basics of machine learning with a short look into the history of machine learning and artificial intelligence followed by a basic introduction to classification and regression tasks, unsupervised and supervised learning and the importance of data. Machine learning is connected to climate science, in particular the detection of atmospheric blocking.

History of Machine Learning and Artificial Intelligence

The powerful and simple mechanics inside the human brain inspired computer scientists in the early 1950s to use a similar mechanic to the neurons inside a human brain. The Dartmouth workshop in 1956 coined the term artificial intelligence for the field of computers learning and thinking similar to processes in the human brain. In the year 1956, Frank Rosenblatt invented the basic perceptron, a single unit with inputs, activation functions and an output [11]. The field quickly grew into more complex setups and algorithms to increase the capacity of early models. Due to lacking compute power and the inability to scale artificial intelligence and machine learning suffered periods of low funding and low research interest. In the 1980s new concepts such as neocognitron by Kunihiko Fukushima which would later evolve into the now widely used convolutional neural network (CNN) improved performance and capabilities significantly. The sharp increases in computational power due to the modern transistor and Moores law enabled the general public and businesses to start training their own models and increased innovation greatly. With the switch to graphical processing units (GPU) the capabilities skyrocketed and modern neural architectures and training processes became plausible [12, 13]. Nowadays, machine learning and artificial intelligence is used in a large portion of new applications and systems. Businesses employ artificial intelligence models to increase productivity or automate tasks hard for traditional algorithmic solutions.

Concepts of Machine Learning

Machine learning relies on data to predict patterns and similarities in new and unseen data. To detect patterns, the model is trained on huge so-called training datasets, learning the inherent structure. Data is split into input features and labels that should be predicted or classified by the machine learning model. During training the model uses input data to try and predict the desired output. After each prediction an error function is used to calculate the distance between the models output and the desired output. This error is used to update weights inside the model to reduce the error between the predicted output and the actual label. To do this, a specific loss or optimization function is used that implements gradient optimization algorithms.

For the actual learning of patterns and connections in the data the concept of neurons is used. The neuron is inspired by the biological neuron inside human brains and mimics their functionality. A neuron is the smallest part of a machine learning model and is used to generate an output of a non-linear function based on all inputs. It consists of an arbitrary amount of inputs, weights for each input, a summation function, an activation function and a bias weight. The importance of each input of a neuron is decided by the specific weight applied to the input value. During calculation, the neuron sums all weighted inputs and adds the bias value. Afterwards the activation function is applied to generate the non-linear output which is propagated to the next layer.

Neural networks use multiple layers of neurons, including an input layer, multiple hidden layers and the final output layer. Neurons on each layer are connected to all outputs of the previous layer, which ensures that neurons can act on all available input data to detect patterns. By utilizing multiple hidden layers, especially during deep learning, models can learn and generalize better, increasing accuracy and convergence. Even though single hidden layers can theoretically represent the same functionality as deeper networks, practice has shown the advantages of utilizing many hidden layers during model construction. To train the final neural network, an algorithm called backpropagation is used. Backpropagation uses derivatives of each layer to assign weights the corresponding share of the final error. This process of propagating the final error through each layer and neuron allows the model to update every weight according to the error, resulting in better predictions and decreased error the next time. This process is repeated on the training data until the model achieves

the desired output and accuracy [14].

All machine learning algorithms rely on the dataset in use. With complex algorithms and models, the need for a high quality and quantity of data increases. Especially for supervised learning tasks, the availability of data is often one of the main issues in advancing model performance, as the labeling of data samples is highly dependent on humans. There are special techniques to train advanced models on smaller datasets such as transfer learning or data augmentation but the basic rule that more data is linked to better performance of an adequate model holds [15]. For fields with large amounts of data, machine learning is especially interesting to find patterns that are not apparent and can be missed by human analysts.

2.2.1 Unsupervised and Supervised Learning

Unsupervised Learning

Unsupervised learning is used to train models on large datasets without known output labels. During the learning process, the model adapts to the underlying structure and patterns of the data without the guidance of the output labels. Tasks for unsupervised learning include clustering input data, reducing data dimensions, removing noise, upsampling data, or generating outputs that resemble specific training data. Algorithms used in unsupervised learning include k-means clustering, principal component analysis, autoencoders, generative adversarial networks, and variational autoencoders [14].

During training unsupervised learning algorithms calculate an error based on input data modifications or restrictions in the design of the model. For example, autoencoder models are trained to reconstruct the input data with a restriction in model size to learn to compress the data into a smaller representation without losing detail. Other models, such as image transformers, train by removing and reconstructing parts of the input data to learn the inherent structure of the data [14].

Supervised Learning

In supervised learning a model is trained on a dataset with already known output values, which have to be collected automatically or labeled by humans. Due to the known output, the model is trained to predict the mapping between input and output data, using an calculated error. According to this error, the weights of the model are adjusted accordingly, reducing future error for similar input data. Supervised learning is used by many machine learning algorithms and models including support vector machines, linear regression, k-nearest neighbor, decision trees, and convolutional neural networks.

Exploring all of the supervised learning algorithms used is beyond the scope of this thesis. As this thesis evaluates the possibility of using convolutional neural networks and random forest classifiers for the detection of blocking, both algorithms are introduced later.

Example tasks for supervised learning include image classification, speech recognition, or regression tasks [14].

Regression

In machine learning, regression tasks involve models or algorithms predicting numerical values from given input data. For linear regression, this is equivalent to the model predicting a function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, where n is the number of input features. The output value for the specific regression task might represent any quantitative measure, such as dollars in an insurance payout. Different types of regression, such as linear, multiple linear, or logistic, are used to predict different relationships in data [14].

While linear regression can be effectively solved using the least squares method under ideal conditions, machine learning offers additional tools for the case where those conditions are not met. For example, least squares assumes that there is a linear relationship and that errors are normally distributed. In real-world data, this is often not the case. Machine learning approaches, particularly those utilizing regularization, can handle real-world data better, providing models that are robust to overfitting, capable of feature selection, and designed to tackle complex or high-dimensional datasets. Overfitting refers to machine learning models starting to memorize the training data and not generalizing further, leading to almost perfect training and worse validation performance. These advantages increase the potential for machine learning techniques in regression tasks across various sectors, from finance to healthcare, wherever predictive accuracy and model generalizability are important [14].

Classification

Classification tasks train a model to correctly choose one of k categories to assign to the y dimensional input data. To do this the model is fitted on specific input features which can be of various dimensions and formats, including sequences (voice), data points, or images. Classification tasks produce a k dimensional output vector corresponding to the probabilities for class $n \in k$ at position n in the vector. This corresponds to the model predicting the function $f : \mathbb{R}^y \rightarrow (0, 1)^k$. For example, a classification task with 3 different classes ($k = 3$) and input feature dimension of 2 ($y = 2$) results in a model predicting the function $f : \mathbb{R}^2 \rightarrow (0, 1)^3$. The model output is a probability tensor representing the class probability for the input data (Output tensor (0.8, 0.2, 0.01) represents a classification of class 1). Other variants of classification models predict the following function $f : \mathbb{R}^n \rightarrow \{1, \dots, k\}$ where the output $y = f(x)$ is directly corresponding to the chosen class [14].

Object and image recognition are examples of classification tasks. Models are trained to detect specific real-world objects in images supplied in 2D maps with either grayscale or RGB channels. A popular example used for training image recognition models is ImageNet, a labeled image dataset with 1000 different categories and over 1 million training images [16].

This thesis evaluates the possibility of supervised learning algorithms to detect blocking events reliably. This is a subset of classification tasks called binary classification, where the output classes always correspond to 0 or 1. Output class 1 corresponds to the presence of a specific trait in the input data, class 0 corresponds to no presence. In the example of atmospheric blocking detection, a model output of 1 corresponds to a blocking event present in the input data.

Random Forest Classifier (RF)

Random forest classifiers (RF) utilize a forest containing multiple decision trees simultaneously to alleviate most of the shortcomings of singular decision trees. Contrary to them, random forests utilize bootstrapping, bagging, and feature selection to increase generalization and accuracy. RFs generalize by fitting multiple decision trees on different subsets of the available data and averaging the output accordingly. During bootstrapping, the dataset is divided into multiple subsets designed to fit one decision tree. The subset entries are chosen from the entire dataset at random with replacements to allow multiple rows in the bootstrapped dataset to be equal. For each bootstrapped dataset, an out-of-bag dataset is created with all rows not included in the bootstrapped one. This is usually about $\frac{1}{3}$ of the original rows. This out-of-bag dataset can be used to evaluate the error and accuracy for each decision tree in unseen data during fitting.

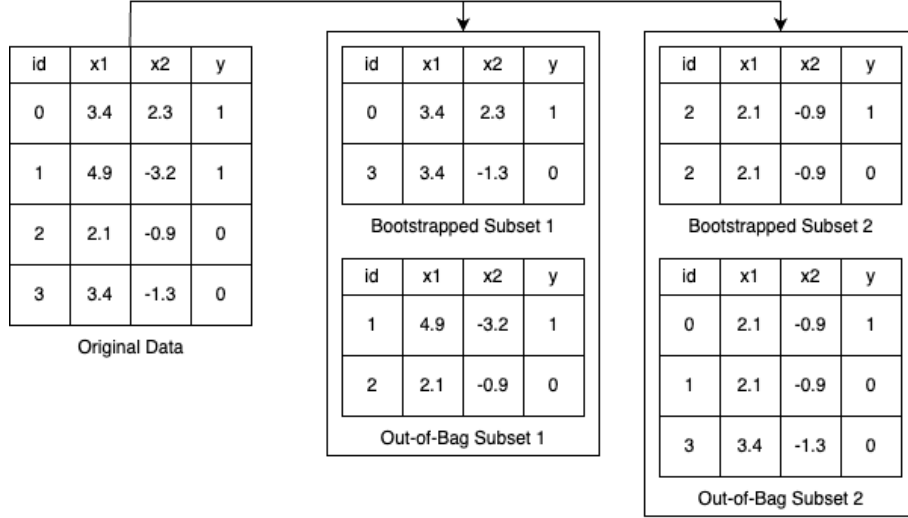


Figure 2.4: Two bootstrapped datasets chosen randomly from the original data. Due to the bootstrapping, both subsets only contain a subset of the original data, possibly missing rows or containing duplicates. The out-of-bag subset, which includes all rows not used to create the bootstrapped subset is depicted below. The bootstrapped data is used to train decision trees.

When fitting the decision tree to the bootstrapped datasets, a random subset of the available features is chosen to stop the model from relying on single features and overfitting. The functionality and creation of decision trees is out-of-scope for this paper but is greatly explained in *Deep Learning* [14] and *An Introduction to Statistical Learning* [17]. Finally, the classifier is run on input data and the output class is chosen by the majority of all decision trees in the random forest.

Convolutional Neural Networks (CNN)

Convolutional neural networks also known as CNNs are neural networks designed to work well on gridded input data. They employ the mathematical convolution operation to work on various input data lengths. This can include 1D time interval data or 2D gridded data such as images. CNNs are comprised of multiple layers including one or more so-called convolutional layers.

The convolution layer uses a special kind of mathematical convolution operation

$$s(t) = \int_k^t x(k)w(t-k) dk$$

This special kind of convolution is called discrete convolution and works on discrete input data, which can be image pixels or specific time interval data.

$$s(t) = (x * w)(t) = \sum_{k=-\infty}^{\infty} x(k)w(t-k)$$

The convolution operation applies a so-called kernel (in this case k) on the input data (in this case t) to produce the function output, the feature map. This kernel can be a 1D or 2D grid and is learned during model training [14].

For blocking detection the input is a 2D grid with measurements of meteorological variables including geopotential height and sea level pressure. Convolutional neural networks provide a good fit for blocking detection, as features can be spread across the grid and a global

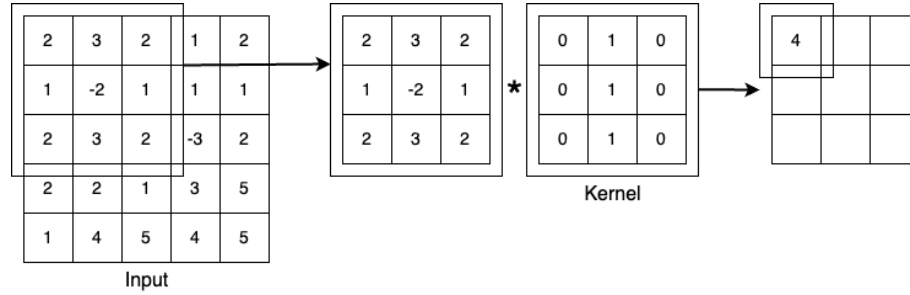


Figure 2.5: A single convolution operation on a 2D input grid with a kernel of size 3x3. The output feature map is size 3x3 with an input of size 5x5. The first convolution is applied on the top-left corner of the input data, selecting an input area equal to the size of the filter used during this operation (3x3). During the convolution, the filter and input area are multiplied element-wise and summed up to produce the convoluted output in the next layer. In this example, the middle row is summed up to produce an output value of 4 ($3+(-2)+3$).

structure has to be taken into account to detect specific events. Convolutional layers excel at learning robust patterns to classify data correctly even with small shifts in the input data.

2.2.2 Linking Atmospheric Blocking and Machine Learning

This thesis evaluates a supervised machine learning approach for detecting a specific phenomenon, atmospheric blocking, with the potential of applications inside multiple climate models and long-term trend analysis. Machine learning can aid in the detection of atmospheric blocking in multiple areas. Hybrid models, that combine traditional numerical and index-based detection methods with supervised or unsupervised models can rely on a broader context to increase performance. Blocking indices are based on specific meteorological variables and regions, which can be extracted and selected by an underlying model. A replacement of traditional detection methods is also possible. Models trained on recorded observational data combined with climate model data are capable of detecting global patterns in multiple meteorological variables. Previous research by Thomas et al. [4] used an unsupervised self-organizing map approach with final clustering to reach state-of-the-art performance on multiple datasets. As a baseline, Thomas et al. created an expert-labeled dataset, which will be used by this thesis to evaluate the application of supervised machine learning for the detection of atmospheric blocking.

2.3 Related Work

The following sections introduce similar and related work in the fields of climate science and machine learning, including different data sets and machine learning approaches for classification and related work regarding atmospheric blocking and blocking detection.

Machine Learning in Climate Science

Due to the large amounts of observed and simulated data, climate science is a prime example for the adoption of machine learning. Applications can range from specific function solvers, replacement of model components, to full replacement of traditional numeric and knowledge-based approaches for solving problems. Due to the sharp increases in computing capabilities in the last 10 years, more areas in climate science look to machine learning solutions as future replacements to older systems and models [18]. Especially in climate change science, the applications for machine learning are apparent. The range of tasks that

benefit from machine learning include data analysis, forecasting, simulations, efficiency improvements, maintenance predictions, inventions, and more. Global climate tasks are inherently connected to large datasets, which are hard or impossible to analyze by humans [18]. Machine learning models can assist the analysis of collected data from satellites or monitoring stations across the globe to provide researchers with faster and deeper insights. In forecasting and simulations, machine learning can greatly increase model accuracy and capabilities. In the fight against climate change, machine learning will greatly increase efficiency in industries with high carbon emissions [19]. The field of object identification in bigger models is also growing at a fast rate. This includes detecting specific phenomena in the model data or supplementing the model data to improve performance during the original task. Supervised and unsupervised learning approaches have been explored, with a major limitation being the required labeled dataset during supervised training [18]. During this paper a machine learning approach for the detection of blocking events with newly introduced labeled data is evaluated and the potential for machine learning in this area of climate research presented.

ClimateNet

A similar example of the application of machine learning in climate science is ClimateNet [20]. The research paper about ClimateNet focuses on the pixel-level identification of extreme weather events, specifically tropical cyclones and atmospheric rivers. With the recent success of deep learning, a similar approach was chosen. Deep learning models require large amounts of accurately labeled data that are often not available. The research team introduces ClimateNet - an expert-labeled and community sources dataset for classifying tropical cyclones and atmospheric rivers in recent climate model simulation data. The research paper continues to fit a state-of-the-art deep learning model to the ClimateNet dataset and apply the model to historical climate data and future climate change simulations and presents an accurate segmentation of atmospheric rivers and tropical cyclones. The research team concludes that a transfer learning approach with deep learning can be applied to a broad range of different tasks and datasets, including observational, reanalysis, or simulated data [20].

Clustered Weather Patterns

In a similar paper published in 2020, a deep learning approach using convolutional neural networks for the prediction of clustered weather patterns is used. With no labeled data available in the research domain the paper proposes an automatic unsupervised clustering approach to generate labeled and clustered data groups. The final clusters are used to train a convolutional neural network to predict the label in advance with high accuracy. The paper concludes by comparing the trained convolutional neural network to logistic regression, increasing accuracy significantly. Finally, the paper proposes the use of machine learning, in particular convolutional neural networks, for a wide range of domains with complex patterns in input data and outlines the possibility of unsupervised clustering for dataset generation [21]. “The scaling that is found [during training] [...] suggests that the amount of data currently available from reanalysis since 1979 can be enough to successfully train an accurate CNN for applications involving daily large-scale weather patterns.” [21]

3. Data and Methodology

This chapter introduces the underlying data, scoring, and preprocessing pipelines used during the evaluation. For machine learning applications, the training and testing data are one of the major contributors to performance and therefore should be available to researchers reproducing the results. No closed-source datasets will be used during the thesis, and dataset abbreviations are clear and concise. The sources, download links, and access dates can be found in the appendix. Datasets and the teams behind them are shortly introduced. Furthermore, this chapter introduces the scoring metrics and training pipeline used to evaluate models and compare results to previous blocking indices and detection methods.

3.1 Used Datasets

Climate datasets are created from multiple different sources and systems. In this study, a reanalysis dataset is used to give a good representation of the real atmospheric conditions. A reanalysis dataset is created by collecting measured data points from meteorological balloons, ground stations, aircraft, satellites, and more. All collected data points are used with a weather model to interpolate measurements to a grid and generate a global coverage of multiple meteorological variables with a specific resolution. With state-of-the-art computer models this leads to very accurate data with global coverage based on observations and measurements [22]. This is the same process used for the observational dataset used during this study, ERA5.

Simulated model data are based on Earth system models with a specific configuration. This simulation can run without background changes and long-term trends such as CO_2 increases due to human fossil fuel consumption. State-of-the-art earth system models combine many modules to simulate interactions between specific land, sea, or atmospheric processes. This can include carbon and nitrogen cycles, stratospheric interactions, coupling between ocean and land, and more [23]. Multiple earth system models often run in ensemble configurations, such as CMIP6, to produce the highest quality of simulation data [24].

ECMWF Reanalysis V5 (ERA5)

European Centre for Medium-Range Weather Forecasts (ECMWF) started publishing reanalysis datasets in 1999, called the ERA-15 project, with approximately 15 years of data. In 2019 ECMWF published their newest ERA5 reanalysis dataset, replacing the 19-year shorter and less accurate ERA-Interim reanalysis dataset.

“ERA5 is the fifth generation ECMWF atmospheric reanalysis of the global climate covering the period from January 1940 to the present. ERA5 is produced by the Copernicus Climate Change Service (C3S) at ECMWF.” [5]

ERA5 provides hourly estimates of meteorological variables from land, oceans, and atmosphere gridded across the globe. With a resolution of $0.25^\circ \times 0.25^\circ$ the reanalysis has a fine grid that is further split up into 137 height levels for atmospheric variables.

For this study the ERA5 dataset for geopotential height at the 500hPa and mean sea level pressure are used (more about the meteorological variables is available in Section 2.1). The data is preprocessed for according to the pipeline defined in Section 3.3 and used during training and testing. The datasets are freely available to download at the ERA5 project website [5] for all years from 1940 to 2023 and specific months and days.

UKESM1-0-LL Pre-Industrial Control Simulation (UKESM)

The UK Earth System Modeling Project (UKESM) is a science project in the United Kingdom focused on creating, applying, and maintaining UK Earth systems models in climate and Earth system science. The UKESM program represents the United Kingdom’s national capabilities in Earth modeling to deliver estimates for the future or provide accurate data for scientific projects. The project is spread across the UK, with scientists from NERC research centers and the Met Office Hadley Centre contributing [24]. The UKESM project contributed to IPCC AR6, the sixth assessment report by the Intergovernmental Panel on Climate Change, by running a full set of CMIP6 simulations, which are available on the official UKESM website. CMIP6 is a global collaboration project that coordinates climate model experiments to improve understanding of climate dynamics, variability, and responses to external factors. Further simulations are available, including the UKESM1-0-LL and UKESM1.1 model configurations. The project teams aim to help scientists and researchers better understand the phenomenon of the Earth system, the evolution of the climate system during the past centuries, and investigate mitigation options for climate change [23, 24].

For this thesis, a UKESM dataset from the UKESM1-0-LL model is used. This dataset was created in 2018 and contains many atmospheric, land, aerosol, ocean, and ice variables. This thesis is using a subset with 101 years of length for geopotential height at 500hPa and mean sea level pressure. UKESM1-0-LL is a model running in a so-called pre-industrial control simulation, referring to a period generally accepted to be pre-industrial without the effect of the Industrial Revolution on the Earth system. This simulation is therefore not connected to real dates but will be timestamped for ease of reading. The UKESM dataset uses a 360-day year and imaginary time range from 05.27.1960 to 04.09.2060. The dataset is preprocessed according to Section 3.3 before being used during training and testing [25].

Ground Truth Dataset (GTD)

The ground truth dataset was created by Thomas et al. [4] during a research paper in 2021. It contains data including 41 years of ERA5 data and 101 years of UKESM data with daily labels for the presence of summer blocking events over Europe.

Thomas et al. created the labeled dataset by using blocking experts to label date ranges by hand without relying on any blocking indices or detection methods. To do this, the dataset was first cut to the so-called JJA date range, corresponding to the summer months June-July-August. The date range was extended into both directions with 4 days, which includes May 27. to 30. and September 01. to 04.

After the date trimming to summer months, the dataset was split into five-day periods, as the duration for blocking is defined with a minimum of five days. These five-day data periods were then labeled by experts according to the presence of atmospheric blocking in the data. If a sample was successfully detected as blocked, all included days receive a positive label. Once all periods from 41 years of ERA5 data and 101 years of UKESM data have been labeled and classified, the final blocking dataset is constructed. For this, every day in the time frame receives a positive label (1) if the day is in a previously labeled period as a block. If the day is not contained in any blocked period, the label negative (0) is applied. With this approach also longer periods with blocking are correctly classified in the final dataset [4].

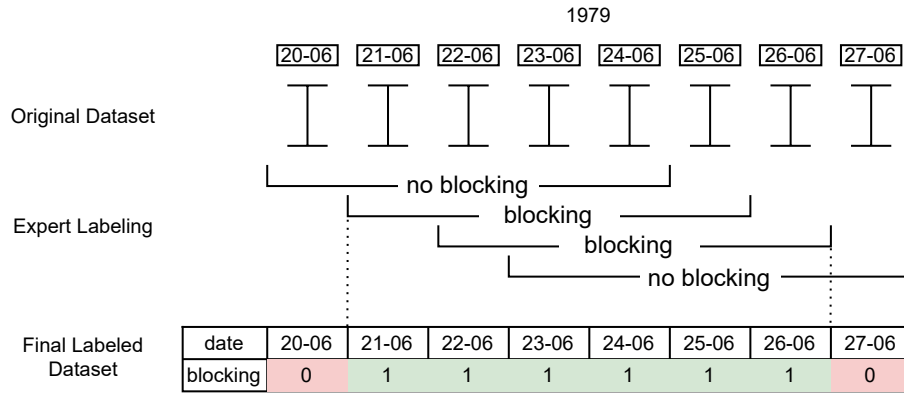


Figure 3.1: Labeling used by Thomas et al. to create a daily expert labeled dataset for ERA5 and UKESM. The original dataset is labeled for blocking occurrences with a minimum duration of five days by experts. For blocking events, all containing days receive a positive label in the final labeled dataset, creating a daily blocking label. With this approach, blocking events longer than five days are correctly labeled, with all containing days receiving a positive label.

3.2 Tools and Libraries

Data Tools

For the preparation and handling of data during multiple steps in this thesis, specific tools were used. The following section includes all tools not considered well-known.

Network Common Data Format (NetCDF)

The Network Common Data Format (NetCDF) is a machine-independent binary data format maintained by the Unidata program at the University Corporation for Atmospheric Research. The first netCDF data format was released in 1990 and is still heavily in use today. It is superseded by netCDF4, the current version, in 2008. NetCDF is used for sharing and working with multidimensional data and can store metadata together with data dimensions to provide a short description to users [26]. As part of this thesis, netCDF is used for all datasets including downloads of ERA5 and UKESM. The GTD by Thomas et al. [4] is also available for download in the form of netCDF files.

Climate Data Operators (CDO)

Climate Data Operators (CDO) is a command line tool to manipulate and organize climate datasets. CDO was originally developed in 2003 by Uwe Schulzweida at the Max Planck Institute for Meteorology and is now maintained by scientists from the Max Planck Institute. The tool supports multiple common data formats including netCDF 3 and 4. There are hundreds of different operators available that can be applied to the entire dataset or specific subsets. This includes simple statistical and arithmetic functions, data selection, interpolation, and more. CDO is free to use and available for download on the official Max Planck Institute homepage [27].

Xarray

Xarray is an open-source Python package that provides functionality for datasets in various formats, including netCDF 3 and 4. Xarray enables working with multidimensional labeled data preserving relations between data points and labels through complex operations and transformations. With an interface similar to the popular Python pandas package Xarray enables interoperability with all major scientific packages including NumPy, Matplotlib,

and pandas. This thesis uses Xarray heavily during data preprocessing and final dataset setup. Working with multidimensional datasets with Xarray and netCDF4 allows fast and correct iteration through time labels and application of Xarray operations [28].

Model Training & Fitting Tools

During training and fitting, machine learning frameworks provide an abstraction layer from the underlying math and create environments to build, train, and test models easily and efficiently. For simpler models, specific frameworks provide out-of-the-box solutions, and there is no need to build a pipeline and model configuration. For complex models, frameworks with low-level access are necessary to build custom training setups and customize existing models.

PyTorch

PyTorch is a deep learning framework originally developed by Meta AI and released in 2016. It is based on the Torch machine learning framework released in 2002. With a Pythonic way of working and high performance during model training, PyTorch established itself as the go-to framework for machine learning research and applications. PyTorch allows users maximum customizability and access to low-level functionality for complex models and training setups [29]. This thesis utilizes PyTorch for data loading, data structuring, and training for convolutional neural networks. The pipeline for data and models is also set up with PyTorch.

Scikit-Learn

Scikit-learn was first released in 2010 and continued development as an open-source machine learning library. Scikit-learn, also called sklearn, integrates a wide range of machine learning algorithms including support vector machines, random forest, linear regression, and more. The library provided unsupervised and supervised algorithms for medium-sized projects without relying on specific configurations or complicated setup procedures. Sklearn focuses on providing a high-level API that is accessible and usable for non-specialists [30]. Sklearn provides good performance with great usability and is used for the random forest classifier and hyperparameter grid search. There are also some utility functions from sklearn used in the CNN training pipeline.

3.3 Data Preprocessing Steps

To prepare the data sets for final training and testing, they must go through the preprocessing pipeline. During this process, the data is modified and unified to ensure that the machine learning model can learn patterns and similarities across the datasets. This involves several steps, which are explained in the following chapters. After preprocessing, all datasets follow a common grid and range of values.

During **re-gridding** the ERA5 and UKESM datasets are scaled to fit the same common grid. This is important for the final model transferability between datasets. Furthermore, this reduces the amount of input data without losing important details and accuracy. For re-gridding, the datasets using bilinear interpolation, CDO with the corresponding `cdo remapbil,<grid> <infile> <outfile>` operator is used.

For ERA5 the original grid is $0.25^\circ \times 0.25^\circ$ which is upscaled to a common grid of $1^\circ \times 1^\circ$. UKESM model data is gridded at $1.875^\circ \times 1.25^\circ$ and is therefore down sampled to the common grid.

```

1 gridtype = lonlat
2 xsize   = 100
3 ysize   = 45
4 xfirst  = -44.5
5 xinc    = 1
6 yfirst  = 30.5
7 yinc    = 1

```

Listing 3.1: Grid file used with CDO to generate the desired latitude and longitude grid (45×100). CDO accepts this grid file as input to produce the final netCDF file with re-gridded values.

For summer European blocking events, only a small subset of longitude and latitude are relevant for classification. ERA5 and UKESM datasets are cut to the longitude range $-45, 55$ and latitude range $30, 75$. This corresponds to a **longitude and latitude window** over Europe including western Russia and large parts of the Atlantic Ocean up to Greenland. The data operations are performed by Copernicus for ERA5 directly [5] and with CDO for UKESM.

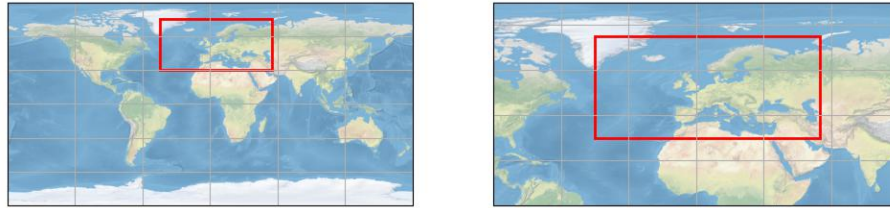


Figure 3.2: Longitude and latitude bounding box used during data preprocessing. Mapped against the entire earth (left) and a cropped northern hemisphere (right). The bounding box is for longitude values between $-45, 55$ and latitude values between $30, 75$.

After geographically processing, the data is cut to the desired time frame. For summer blocking anomalies only the **summer months** of data are needed during classification. Including more data than necessary would create an unbalanced dataset with unnecessary negative data points during months outside of summer. During this preprocessing step, ERA5 and UKESM datasets are cut to the so-called JJA period, which consists of the summer months of June, July, and August. A four-day extension is added on either side to create complete five-day periods from June up to the end of August. All date cuts are done with CDO in the netCDF data files. The final datasets contain the following daily entries:

- ERA5
 - 41 years from 1979 to 2019
 - yearly from May 27. to September 04.
- UKESM
 - 101 years from arbitrary 1960 to 2060
 - yearly from May 27. to September 04.

After removing unnecessary months from the dataset, seasonal trends from the meteorological variables used need to be removed. The general idea behind the de-seasonalizing approach used in this thesis is to remove the yearly variation on each day of the year. Furthermore, this approach has to be specific to a single grid cell, as the yearly fluctuations are dependent on longitude and latitude. For this, a rolling mean is created for each day of the year, and each grid cell that is subtracted from every grid value. With this approach, a

good de-seasonalization is achieved without destroying anomaly data due to a high rolling window of 10 years.

For this explanation, a de-seasonalizing run with ERA5 data is assumed. The de-seasonalization function takes a dataset with dimensions $(4018 \times 45 \times 100)$ as input which is returned without seasonal variations. A daily climatology variable is created by grouping the dataset by the corresponding time of year and calculating the mean for every group across all years of the input dataset. This variable is created equally for each cell and stored in a map with dimensions $(n \times 45 \times 100)$ with n corresponding to the days per year in the input dataset. This map is then subtracted from each sample in the dataset to achieve the desired de-seasonalization. Finally, the dataset is returned with the newly calculated values for each day and grid cell. The Python function for de-seasonalizing can be found in the Appendix 6.4.

The de-seasonalized data has no yearly influence but still contains long-term trends. This has a different impact on different meteorological variables, as MSLP is quite resistant to change in the atmosphere. Contrary Z_{500} is heavily influenced by atmospheric changes such as continued CO_2 emissions from human activity. Data with trends not removed from the training dataset result in the model having a hard time learning patterns and features as the input data has a higher or lower mean throughout. This trend is removed by **linear de-trending** the available data with numpy.

First, a list x is created with numbers ranging from 0 to the length of the data. This will be used as inputs to the linear function and represents a specific day in the dataset. The mean is calculated across all data points along the time dimension. Then the coefficient for the linear trend function is calculated with numpy and finally, the de-trending function is set up. An array is created for each day in the dataset with a corresponding de-trending value for each grid cell. Finally, for every day the de-trending function values are subtracted from the actual values in each grid cell. The final de-trended data is returned to be used in the next steps.

```

1 def detrending(dataset):
2     x = np.arange(len(dataset.time))
3     mean = np.mean(dataset[variable].to_numpy(), axis=(1,2))
4     coef = np.polyfit(x, mean, 1)
5     poly1d_fn = np.poly1d(coef)
6
7     poly_array = np.array([np.full((len(dataset[lat]), len(dataset[lon])),
8     poly1d_fn(x)) for x in range(len(dataset.time))])
9     detrended_ds = dataset - poly_array
10    return detrended_ds

```

Listing 3.2: Python function used for de-trending of all input datasets. Dependencies for this function include Xarray and NumPy. A linear de-trending function is fitted to the data and subtracted for each data point to remove the global linear trend.

After de-seasonalization and de-trending, the data is normalized. Normalization ensures a similar range of values is used between different datasets, therefore helping the model learn variations in the data. For normalization of the data the so-called *z-score* is used. The z-score represents a data distribution with a mean of zero with a standard deviation of one [14]. The de-seasonalized and de-trended data is divided by the standard deviation σ . The resulting values have a mean close to zero and a standard deviation of one.

```

1 data[variable].data = data[variable].data/np.std(data[variable].data,axis
2 =0)

```

Listing 3.3: Normalization using the standard deviation on the dataset. For this the numpy std function is used.

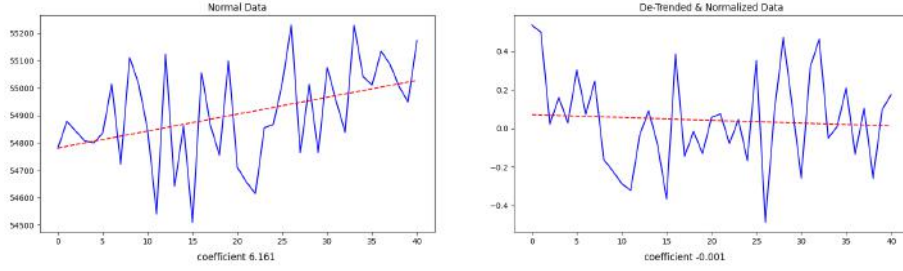


Figure 3.3: Mean data plotted for a single cell of the data before (left) and after de-trending and normalizing. The linear trend coefficient on the data before de-trending is 6.1 and on the de-trended data 0.

In the final preprocessing step the dataset is reshaped into five-day periods. This is necessary to create data samples with a shape of $(5 \times 45 \times 100)$, enabling the model to classify a blocking event with a minimum duration of five days for every input sample. For the transformation, Python slicing is used with duplicate last elements to allow a complete transformation into five consecutive days.

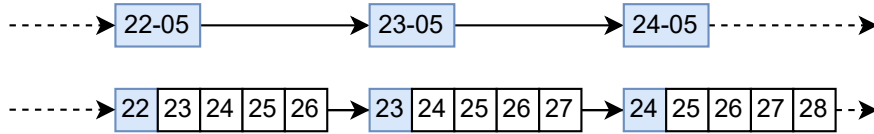


Figure 3.4: Data transformation to include five consecutive days for all samples to allow independent blocking detection on each data point. For this, every day is concatenated with the following four days to create the necessary dimensions.

3.4 Machine Learning Pipeline

HAICORE

For computation during data transformation, model training, and testing the **H**elmholtz **A**I **C**ompute **R**esource project HAICORE is used. HAICORE is a part of HoreKa, a tier 2 supercomputer at Karlsruher Institute for Technology (KIT). HoreKa consists of almost 60.000 Intel processors and 668 NVIDIA A100 GPUs connected to a high-performance network. HAICORE consists of 16 nodes with different assigned resources. A login node dedicated to connections for end users, 12 nodes with 4 attached NVIDIA A100 GPUs, and 3 nodes connected to 8 NVIDIA A100 GPUs [31].

For training and hyperparameter search a compute node with 4 NVIDIA A100 GPUs is utilized and for final training, a single NVIDIA A100 GPU is used. The performed hyperparameter optimizations greatly benefit from computational capabilities, with HAICORE providing a great performance increase from traditional hardware. Nonetheless, HAICORE still limits job resources to a maximum of 4 NVIDIA A100 GPUs, limiting the potential for large-scale hyperparameter search further described in Section 6.1.

3.4.1 Model Choice

The following sections introduce both algorithms chosen to evaluate supervised learning for the detection of atmospheric blocking. A wider selection of supervised algorithms for blocking detection is discussed in Section 6.4.

Random forest classifiers are used as a simple supervised learning algorithm for comparison between supervised methods. They are almost resilient to overfitting by utilizing

multiple decision trees, the so-called forest, simultaneously and deciding the final prediction by majority voting among all trees in the forest. Due to the underlying simplicity, random forest classifiers are easy to fit even on large input data. In the case of atmospheric blocking detection on input data with dimensions $(5 \times 45 \times 100)$ the final input dimension for random forest consist of $5 \times 45 \times 100 = 22500$ features.

For advanced supervised learning algorithms **convolutional neural networks** are chosen. They are designed to work similarly to the human eye and learn to extract patterns and context from two-dimensional input data. They are more complex than random forest classifiers and require careful setup and fine-tuning during training, but can achieve higher accuracy on complex tasks. Convolutional neural networks power state-of-the-art models in image classification, image segmentation, and more. They work great on high-feature data and allow multi-channel input such as RGB images or, in this thesis, five days of blocking data. For this thesis, convolutional neural networks with input dimension $(5 \times 45 \times 100)$ are used. A major downside of convolutional neural networks is the required large datasets for training models on complex tasks to achieve high performance. With limited data availability, this thesis utilizes transfer learning and pre-trained models to alleviate the downside of CNNs requiring large datasets during training. This is explained in further detail in Section 4.3.

3.4.2 Hyperparameter Optimization

Hyperparameters play a crucial role in almost all machine learning models. They can influence model behavior during training by increasing computational cost, training time, and model performance. Regularization hyperparameters are included to find a good generalization without overfitting or underfitting to the data. Some hyperparameters are directly linked to model performance during validation and testing and therefore the right hyperparameters are crucial to achieve good model performance. Some of the hyperparameters often used for optimization include learning rate, amount of hidden units, dropout, weight decay, optimizers, and schedulers. The process of selecting the right hyperparameters is often called hyperparameter search and can be done manually by trial-and-error or through automatic search and optimization algorithms [14]. As part of this thesis two hyperparameter search algorithms, grid search and Propulate, are used.

Grid Search

For a small set of hyperparameters, one of the simplest automatic search algorithms is grid search. The user specifies a set of variables to explore for each hyperparameter, and the algorithms train a model for each unique combination of all options. In the end, the combination with the best performer for the chosen scoring metric is selected with the corresponding hyperparameters.

Grid search introduces new hyperparameters, the values to search for each hyperparameter that need to be specified manually by the user. Often, these values are chosen based on well-performing models on similar tasks and follow a logarithmic scale. For example, a common set of values for learning rate is $\{0.1, 0.01, 10^{-3}, 10^{-4}\}$.

The search on a grid is only possible for a small number of hyperparameters, as the computational cost grows exponentially with the number of hyperparameters [14].

Propulate

For a larger set of hyperparameters, the computational cost with traditional automatic search algorithms such as grid search or random search grows exponentially and is often not feasible even for high-performance compute clusters [14, 32].

The Propulate package and algorithm were developed in 2023 by Taubert, O et al. [32] and are tailored to large hyperparameter searches for complicated optimization problems and modern neural networks.

“Propulate is a [...] [high-performance computing] tailored software for solving optimization problems in parallel. More specifically, it is an asynchronous evolutionary optimization algorithm and Python package with MPI parallelization. This makes it particularly suitable for large-scale hyperparameter optimization and neural architecture search.”[33].

Propulate uses the standard message-passing interface (MPI) for communication between workers. This ensures easy setup, as MPI is a common component in all systems and fast execution due to the simplicity of MPI in comparison to other data storage and communication options. The inherent asynchronous architecture behind Propulate enables a faster search without workers relying on other individuals to finish before starting the next batch of runs. The algorithm is inspired by biological evolution and combines selection, recombination, and mutation during the search. For each generation, genes from previous high-performance runs are incorporated into new candidates, improving performance compared to search algorithms covering the entire hyperparameter space [32, 33].

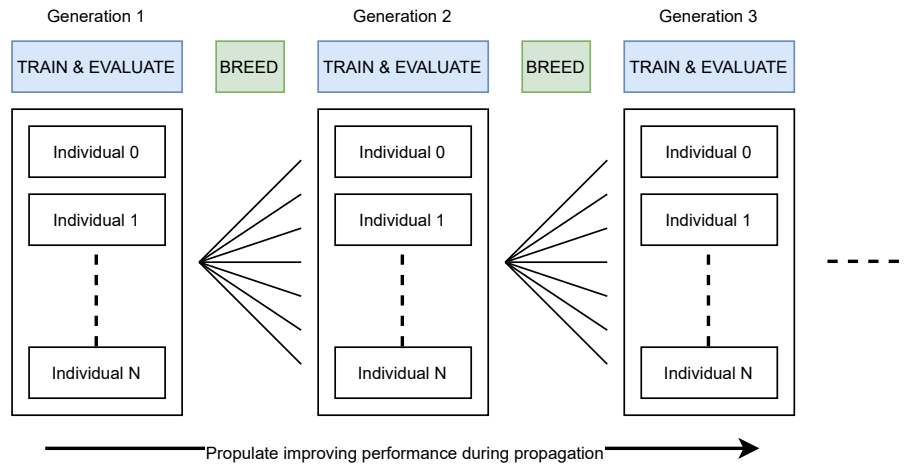


Figure 3.5: Propulate evaluating and breeding individuals during multiple generations. Traits from best performers in previous generations are selected, mutated, and bred to produce increasingly fit individuals in the next generation.

Propulate introduces hyperparameters in the form of probabilities for mutation and mating, number of individuals, emigration policy, and more. For the majority of hyperparameters in Propulate Taubert, O et al. defined suitable defaults in 2023 [32] that are also used in this thesis.

3.4.3 Evaluation Metrics and Validation Methods

Metrics and validation methods play a crucial role in model training, testing, and final results. This thesis applies industry-standard methods to ensure a robust training pipeline and results. For this, a k-fold split in conjunction with cross-validation is utilized. The scoring metric is F1, which is a combination of recall and precision. Finally, after the hyperparameter search the model performance is calculated using a special train/test split on the entire dataset to ensure a robust result even with the small observational and model datasets used in this thesis.

K-Fold Cross Validation

Cross-validation is a hyperparameter validation approach often used for smaller datasets. This increases the amount of validation samples and robustness at the cost of higher

computational requirements. The procedure is based on repeatedly splitting the training data into a validation and training subset, consequently training a model on the subsets and validating the performance. Finally, all splits are combined to generate a mean error and accuracy across the entire training dataset [14, 34]. For k -fold cross-validation the dataset is split into k non-overlapping subsets used consecutively for training and validation. Finally, the performance is evaluated with the average across all 5 model runs.

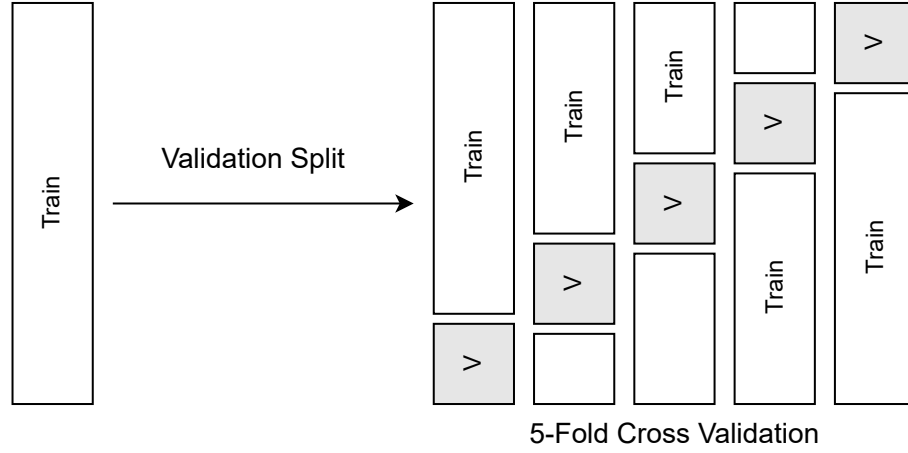


Figure 3.6: 5-fold cross-validation on a training dataset. After the validation split, five equal subsets of training and validation data are created. After splitting, the algorithm is trained on all subsets separately and final results are averaged between all five runs.

Performance Metrics

The accuracy metric chosen is important and will guide all future training and improvement efforts. There are many different metrics with advantages and disadvantages to consider. For this bachelor thesis, a relation score between precision and recall, the so-called F-score is used.

Important for all metrics is the range of possible results in a binary classification task. There are four possible outcomes for an output: true positives, true negatives, false positives, and false negatives. True positives refer to a correct prediction of the positive class and true negatives refer to a correct prediction of the negative class. False positives refer to a model prediction of the positive class while the sample belongs to the negative class. False negatives refer to model predictions of the negative class while the sample should be classified as positive [14].

Precision and Recall

Precision, also called positive predictive value, is the number of true positives divided by the total number of all positive classifications.

$$\text{Precision} = \frac{\text{correct positive classified}}{\text{all positive classified}} = \frac{\text{true positives}}{\text{true positives} + \text{false positives}}$$

Recall, also known as sensitivity, is the number of true positive classifications divided by all positive instances in the data.

$$\text{Recall} = \frac{\text{correct positive classified}}{\text{all positive samples}} = \frac{\text{true positives}}{\text{true positives} + \text{false negatives}}$$

Choosing model architectures and comparing scores with precision and recall is not advised, as a perfect recall can be achieved by classifying every sample with a positive label. Likewise, when only classifying a very small amount of likely samples the model will achieve almost perfect precision. Increasing precision often leads to a decrease in recall and vice versa. Therefore, a relation between precision and recall is used to measure model performance, as a model with high precision and high recall is performing well on the data. Multiple metrics built upon a combination of recall and precision including the F-score, Matthews correlation coefficient, and Markedness can be used [14].

F1 Score

Generally the F-score F_β is the relationship between precision and recall with a β times higher importance of recall. Common values for β are 2 for a higher weight on recall and 0.5 for a higher weight on precision.

$$F_\beta = \frac{(1 + \beta^2) \cdot \text{precision} \cdot \text{recall}}{(\beta^2 \cdot \text{precision}) + \text{recall}}$$

The so-called F1 score is a special case of the F-Score. With a β value of 1, an equal importance is given to recall and precision. The so-called F1-score is the harmonic mean of precision and recall. The F1 score is used to evaluate the performance of all blocking methods during later comparisons.

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Confusion Matrix

A confusion matrix, also called an error matrix, is a specific table visualization of model performance during training and validation. The matrix is used to display the relation between true positives, true negatives, false positives, and false negatives. The diagonal of the matrix always corresponds to correct classifications, in the case of binary classification true positives and true negatives. During training most of the samples should shift towards the diagonal, showing model improvement in classifications. For binary classification, the confusion matrix is 2×2 . For classification tasks with more than two classes, the confusion matrix grows to $n \times n$ with n number of classes for classification. True labels provided by the labeled dataset are listed on the y-axis (left) of the matrix while model predictions are on the x-axis (bottom). A cell always corresponds to the model predicting the class corresponding to the x value of the cell, while the actual data sample corresponds to the y value of the cell. When x and y are the same (the cell is on the diagonal), the classification is correct.

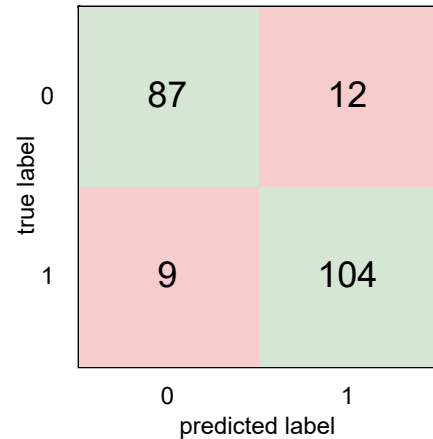


Figure 3.7: Confusion matrix for a binary classification task

Final Train and Test Split

For the final model performance, a separate training and test split is used. This final run aims to generate the most robust model results possible by training and testing on the entire

dataset while maintaining a strict training and testing data split. This is accomplished by training and testing k different models on the data with k number of years in the dataset. During a run, a model is always trained on $k - 1$ years and tested on exactly one year. All testing runs concatenated create a testing result for all k years in the dataset. Performance metrics are then taken on all test predictions and labels to compute the final performance of the model.

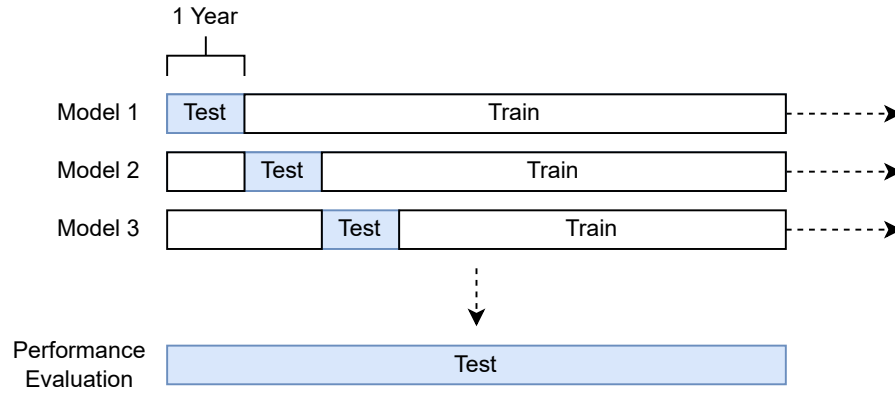


Figure 3.8: Final evaluation setup for random forest and convolutional neural network with k yearly splits and training of k models. Final performance evaluation is done on aggregated test data for the full dataset.

4. Implementation

Links to the Python code, Jupyter Notebooks, and datasets are available in the appendix. The following chapter introduces the data pipeline used during model training and both hyperparameter search and training setups for the random forest classifier and convolutional neural network.

4.1 Data Pipeline

Datasets used for all algorithms are implemented with PyTorch and stored in `model/dataset.py`. They implement the necessary PyTorch dataset interface including the `__init__`, `__len__` and `__getitem__` magic methods. Datasets match labels to all entries during the `__getitem__` magic method, returning the actual data, the blocking label, and the corresponding time. Datasets are important for later use with dataloaders to allow advanced setup of the data pipeline. One important dataset utilized in all later training runs is the `TransformDataset`, which is used during the data augmentation step. It implements the same magic methods with an added transform parameter controlling the image augmentation strength for observational and model data (Listing 4.1).

```
1 class TransformDataset(Dataset):
2     def __init__(self, dataset, transform=None):
3         data, label, time = zip(*dataset)
4         self.data = data
5         self.labels = label
6         self.time = time
7
8         self.transform = transform
9
10    def __len__(self):
11        return len(self.data)
12
13    def __getitem__(self, idx):
14        data = self.data[idx]
15        label = self.labels[idx]
16        time = self.time[idx]
17
18        if self.transform:
19            data = np.transpose(data, (1, 2, 0))
20            data = self.transform(image=data)["image"]
21            data = np.transpose(data, (2, 0, 1))
22
23        return data, label, time
```

Listing 4.1: TransformDataset used for image augmentation pipeline. All custom datasets subclass the official PyTorch dataset and implement the magic methods needed for usage in different functions such as PyTorch dataloaders. For every item returned by the TransformDataset the provided transformation is applied.

PyTorch provides a wrapper for datasets called dataloaders. For model training retrieving samples one at a time from datasets is not practical and leads to bad performance, the dataloader wrapper abstracts the complexity of batch loading and more with a simple iterable API. They additionally provide speedup by using the PyTorch multiprocessing API internally.

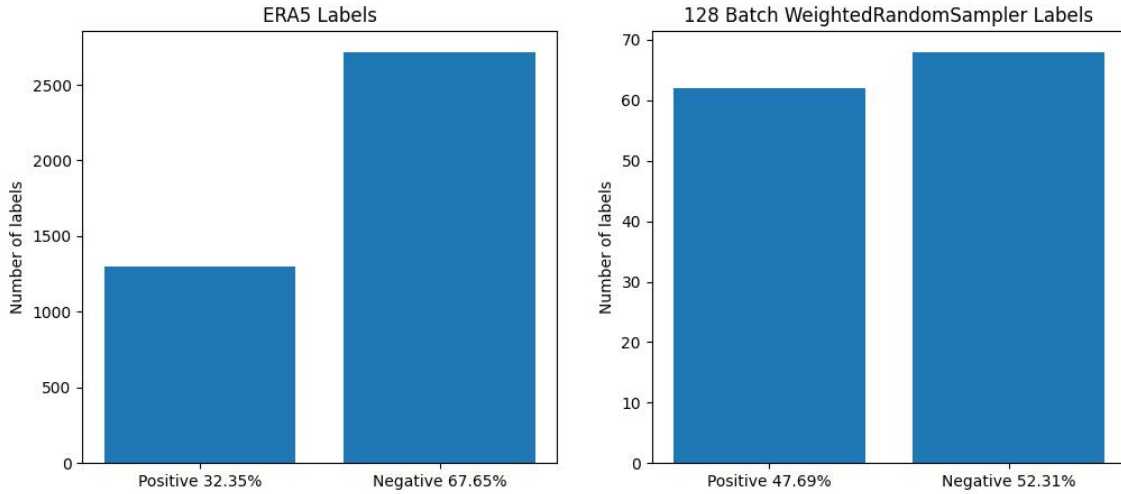


Figure 4.1: Imbalanced labels for ERA5 dataset (left) and balanced label distribution with a 128 batch weighted random sampler (right). The weighted random sampler results in the minority class being sampled more and a resulting batch with a similar amount of both classes to reduce imbalance during training.

Dataloaders also provide an API for utilizing data samplers to change the selection process of data samples. In this thesis, the `WeightedRandomSampler` is utilized for some implementations. This sampler is responsible for applying a specific weight to certain data entries increasing their likelihood of appearing in a specific batch (Figure 4.1). For imbalanced data, samples from the minority class are applied a higher weight compared to samples from the majority class. This process is called weighting and can improve model convergence and performance by providing more examples of the minority class during training.

4.2 Random Forest (RF)

The random forest implementation utilizes a Jupyter Notebook for all steps related to model setup, configuration, and training. The following sections introduce the basic training, hyperparameter search, and testing setup implemented for the random forest classifier.

Training Pipeline

For training the data is first reshaped into the required shape for the random forest classifier. For ERA5 this reshape alters the data from shape $(4018 \times 5 \times 45 \times 100)$ to a shape of (4018×22500) . The input dimension is reduced from the pre-processed five-day period to a one-dimensional feature vector. Reshaping is applied to all datasets resulting in the same $(n \times 22500)$ input size for the random forest classifier.

Grid Search

After data preparation, the parameter grid for the hyperparameter search is set up. This paper optimizes six hyperparameters for the random forest classifier:

- `n_estimators`: The number of trees generated for the forest

- `max_depth`: The maximum depth of each tree in the forest.
- `min_samples_split`: The minimum number of samples needed to split an internal node into multiple nodes.
- `min_samples_leaf`: The minimum number of data points needed to form a leaf node. If a split would produce a branch with fewer samples, the split is not created.
- `bootstrap`: Whether to bootstrap the input data before fitting.
- `max_features`: The number of features to consider for each random subset for a single decision tree.

```

1 param_grid = {
2     "n_estimators": [100, 200, 400, 600],
3     "min_samples_split": [1, 2, 4, 8],
4     "min_samples_leaf": [1, 2, 4, 8],
5     "max_features": ["log2", "sqrt", 0.5],
6     "bootstrap": [True],
7     "max_depth": [None],
8 }

```

Listing 4.2: Parameter grid for random forest hyperparameter search. Bootstrap and `max_depth` are fixed values and not validated through the search process. Values are chosen based on default values and common used values for random forests.

The defined parameter grid and the scoring values (F1-score, precision, and recall) are used to initialize the `sklearn` grid search function. After fitting to the dataset, `sklearn` returns the top performer, with detailed information available for export. The results are recorded and hyperparameter configurations are saved for later use during final training and testing.

Testing Pipeline

For random forest classifier final testing a 41 fold and 101 fold pipeline is set up for ERA5 and UKESM data. Afterward, a random forest classifier is fit for each fold consisting of 40 training and 1 testing year. The testing year is predicted with the final fitted classifier producing the testing outputs, which are appended to the final test predictions. After all folds the performance is calculated on the combined years of testing predictions using `f1_score`, `precision_score`, and `recall_score` from `sklearn.metrics`. For transferability performance between datasets, a forest is fitted on the entire observational or model data and is then used to predict the unseen dataset. Metrics are calculated on the prediction outputs of the unseen data.

4.3 Convolutional Neural Network (CNN)

Convolutional neural networks are implemented in Python and consist of multiple steps. The setup for CNNs is more complicated and computationally intensive than random forest classifiers.

Pre-Trained Models

Training CNNs from scratch is difficult, as they require large amounts of data and computing power.

Transfer learning increases performance by transferring knowledge from a different dataset and task used during training to a similar task, often with a smaller dataset. For most scenarios, abundant training data with corresponding labels is not available. Collecting the required data is expensive and depends on human labor and experts. Transfer learning

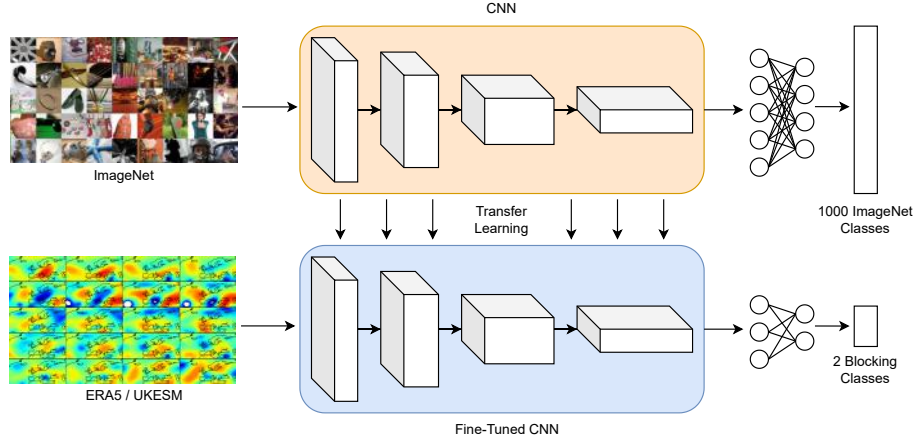


Figure 4.2: Transfer learning from a pre-trained CNN on ImageNet [16] with a final fully connected layer for 1000 classes used during training (top). Transfer learning is applied to re-use weights and fine-tune them with the new, ERA5 and UKESM input data (bottom). The final fully connected network is replaced to accommodate the new binary classification task.

solves this issue by requiring a smaller amount of labeled samples due to previous generalized knowledge in a different but similar task [35].

Large image datasets such as ImageNet [16] provide millions of labeled examples for image classification that can be used to train state-of-the-art models. A widespread approach to building CNN classifiers with limited data availability relies on CNNs trained on ImageNet and fine-tuned on the available labeled data. This approach is based on the principle of CNNs abstracting in the convolutional layers to detect basic shapes and patterns in the data. This can include edges, lines, or similar shapes that are not specific to ImageNet, but also useful for other tasks. For final fine-tuning models can be re-trained fully or only on specific layers. so-called 'freezing' is applied to specific parameter groups belonging to model layers to keep them the same during training. This is especially useful for the beginning layers, as later layers start to be used for specific features of the training data. The final fully connected layer is replaced with a specialized fully connected layer for the specific task required.

This thesis evaluates multiple pre-trained CNNs for detecting atmospheric blocking including ResNet, EfficientNet, and Inception V3, introduced in the following sections. For all models, the first layer is replaced with a different convolutional layer with five instead of three input channels. The original models classify RGB images with three input channels corresponding to red, green, and blue. The input channel number is increased to five, one channel per day in the training data. For atmospheric blocking detection, the input dimensions are described in more detail in Section 3.3.

ResNet

ResNet is short for residual networks, a learning architecture to combat the vanishing gradient issue in deep CNNs. It introduces special layers called residual connections providing a shorter path for the gradient to propagate, by skipping layers. ResNet was presented in 2016 and achieved state-of-the-art performance on ImageNet and comparable benchmarks. ResNet was introduced by Kaming He et al. [36] from Microsoft Research and greatly advanced performance and capabilities of deep CNNs. This thesis evaluates **ResNet18** and **ResNet50** with 18 and 50 layers respectively. The model re-configurations in Listing 4.3 apply to both, ResNet18 and ResNet50.

```

1 conv1_weights = model.conv1.weight.clone()
2 model.conv1 = nn.Conv2d(5, 64, kernel_size=7, stride=2, padding=3, bias=
  False)
3 with torch.no_grad():
4     model.conv1.weight[:, 0] = conv1_weights[:, 0]
5     model.conv1.weight[:, 1] = conv1_weights[:, 0]
6     model.conv1.weight[:, 2] = conv1_weights[:, 0]
7     model.conv1.weight[:, 3] = conv1_weights[:, 0]
8     model.conv1.weight[:, 4] = conv1_weights[:, 0]
9
10 fully_features = model.fc.in_features
11 model.fc = nn.Sequential(
12     nn.Dropout(p=dropout), nn.Linear(fully_features, 1), nn.Sigmoid()
13 )

```

Listing 4.3: Model customization for ResNet18 and ResNet50 architectures. The first convolutional layer is replaced to accommodate inputs with five channels. Pre-trained weights are applied to all new filters. The final fully connected layer is replaced with a dropout layer, a fully connected layer with one output neuron, and a final sigmoid function.

EfficientNet V2

EfficientNet was introduced in 2019 by Mingxian Tan and Quoc V. Le [37] and introduced a new scaling method improving parameter efficiency greatly. EfficientNet models showed competitive performance across the ImageNet benchmarks with substantially fewer parameters compared to similar performing architectures. By lowering parameter count EfficientNet models can significantly reduce computational requirements during training and speed up backpropagation greatly. EfficientNet V2 was published in 2021 by the same authors further improving the model efficiency and scaling. EfficientNet employs multiple advanced techniques to improve performance while lowering parameter counts, such as inverse residual connections and convolutional depth layers [38]. This thesis evaluates two models, EfficientNetV2M and EfficientNetV2S, with different parameter counts for atmospheric blocking detection. The model re-configuration for both M and S models is the same and can be found in Listing 4.3. Dropout is directly applied to both models during initialization.

```

1 weights = model.features[0][0].weight.clone()
2 model.features[0][0] = nn.Conv2d(5, 24, kernel_size=3, stride=2, padding=1,
  bias=False)
3 with torch.no_grad():
4     model.features[0][0].weight[:, 0] = weights[:, 0]
5     model.features[0][0].weight[:, 1] = weights[:, 0]
6     model.features[0][0].weight[:, 2] = weights[:, 0]
7     model.features[0][0].weight[:, 3] = weights[:, 0]
8     model.features[0][0].weight[:, 4] = weights[:, 0]
9
10 fully_features = model.classifier[1].in_features
11 model.classifier[1] = nn.Sequential(nn.Linear(fully_features, 1), nn.
  Sigmoid())

```

Listing 4.4: Model customization for EfficientNetV2S and EfficientNetV2M architectures. The first convolutional layer is replaced to allow five-channel inputs. Pre-trained weights are applied to all new filters. The final fully connected layer is replaced with a fully connected layer with one output neuron and a final sigmoid function. Dropout is directly applied to the model with integrated dropout layers.

Inception V3

Inception models also called GoogleNet were introduced in 2016 by Christian Szegedy et al. [39] and performed exceptionally well in the ImageNet 2015 competition. The Google Brain

team published Inception V3 and introduced the concept of capturing local and global features inside a convolutional layer by utilizing filters of different sizes. This includes 1x1, 3x3, and 5x5 convolutions applied in the same layer. By applying filters in a specific order, the input dimension is efficiently reduced, greatly increasing training speed. With several other efficiency improvements Inception strives to balance computational requirements with high performance [39]. This thesis evaluates Inception V3 for atmospheric blocking detection. For Inception, the input dimensions are fixed to (299×299) , and therefore a resize transform is applied to all input data during training and testing. Dropout is applied directly to the Inception model during setup. The setup, including replacement of the first layer, resizing of the input and dropout integration is implemented according to Listing 4.3.

```

1 weights = model.Conv2d_1a_3x3.conv.weight.clone()
2 model.Conv2d_1a_3x3 = nn.Sequential(transforms.Resize(299, antialias=True),
   BasicConv2d(5, 32, kernel_size=3, stride=2))
3
4 with torch.no_grad():
5     model.Conv2d_1a_3x3[1].conv.weight[:, 1] = weights[:, 0]
6     model.Conv2d_1a_3x3[1].conv.weight[:, 1] = weights[:, 0]
7     model.Conv2d_1a_3x3[1].conv.weight[:, 2] = weights[:, 0]
8     model.Conv2d_1a_3x3[1].conv.weight[:, 3] = weights[:, 0]
9     model.Conv2d_1a_3x3[1].conv.weight[:, 4] = weights[:, 0]
10
11 fully_features = model.fc.in_features
12 model.fc = nn.Sequential(nn.Linear(fully_features, 1), nn.Sigmoid())

```

Listing 4.5: Model customization for the Inception V3 architectures. The first convolutional layer is replaced to allow five channel inputs with a required transform to fit the required input dimension. Pre-trained weights are applied to all new filters. The final fully connected layer is replaced with a fully connected layer with one output neuron and a final sigmoid function. Dropout is directly applied to the model with integrated dropout layers.

Training Pipeline

Convolutional model training is more complex than random forest fitting and therefore requires a training pipeline with more configuration and data preparation. This includes hyperparameter search, data sampling, data augmentation, and more.

Propulate Search

Propulate is a high-performance optimization package designed to enable large, parallel optimization of parameters. With asynchronous evolutionary algorithms, large hyperparameter spaces can be optimized. In this paper, the Propulate hyperparameter search is based on an exhaustive parameter dictionary defining ranges to search in. Due to the self-implemented training and validation function used by Propulate, the search can be extended to hyperparameters not usually used during random search or grid search. For this thesis, CNNs will be optimized by searching the following hyperparameters: Neural architecture (`model`), learning rate scheduler (`scheduler`), loss function (`loss`), dataset sampler (`sampler`), data augmentation pipeline (`augmentation`), learning rate (`lr`), training and testing batch size (`batch_size`), optimizer (`optimizer`), dropout probability (`dropout`) and l2 weight decay (`weight_decay`). The following sections introduce all available options for hyperparameters.

Neural Architecture

Neural architecture refers to the model used during training and testing. Propulate selects one of the previously explained pre-trained CNNs (ResNet18, ResNet50, EfficientNetS, EfficientNetM, or Inception) for the next training and validation loop to evaluate performance differences between all architectures.

Learning Rate

Learning rate is one of the most popular hyperparameters to fine-tune for model performance and influences the weight updates for every training loop. For this bachelor thesis, a range of learning rates is searched starting with a high learning rate of 0.01 to a low learning rate of 0.0001.

Learning Rate Scheduler

Learning rate schedulers adjust the learning rate during training to reach better convergence by fine-tuning the learning rate according to model training progress. For this thesis `lr_scheduler.StepLR`, `lr_scheduler.ReduceLROnPlateau` and no scheduler is evaluated.

- StepLR
 - Reduce learning rate by factor gamma (γ) after n steps
 - $\gamma \in \{0.1, 0.5, 0.9\}$, $n = 10$
- ReduceLROnPlateau
 - Reduce learning rate once loss metric is not decreasing by factor 0.1 with a threshold of 3 epochs

Loss Function

The loss function is used to calculate the training error between the model prediction and the actual label. Propulate evaluates binary cross entropy and weighted binary cross entropy. Binary cross entropy is a loss function designed for binary classification tasks and is widely used in machine learning.

$$H_p(q) = -\frac{1}{N} \sum_{i=1}^N (y_i \cdot \log(p(y_i)) + (1 - y_i) \cdot \log(1 - p(y_i)))$$

Mathematically, binary cross entropy is the negative average of the logarithmic probability of the correct class. This is used during backpropagation and trains the model to assign higher probabilities to the correct class and lower probabilities to the incorrect class. A deep explanation of the binary cross entropy function is beyond this thesis but is explained greatly in *Deep Learning* [14]. For atmospheric blocking a weighted cross entropy loss is additionally evaluated, balancing out the class distribution in the imbalanced data. In weighted cross entropy, samples from the minority class receive a higher weight, which is multiplied by the calculated loss to help the model learn from the minority class. The setup for a weighted binary cross entropy using PyTorch can be seen in Listing 4.3.

```

1 class_counts = torch.bincount(labels.long())
2 class_weights = batch_size / (2.0 * class_counts.float())
3 sample_weights = class_weights[labels.long()]
4 criterion = nn.BCELoss(weight=sample_weights)

```

Listing 4.6: Setup for weighted binary cross entropy loss. The weights are an array with length `batch_size` and pair each data point with the inverse class frequency of the batch.

Dataset Sampler

For data management, PyTorch `DataLoaders` are used. Data loaders use samplers to influence the selection of data samples during training. Propulate is used to evaluate no sampler or `WeightedRandomSampler` to balance out class distribution in the imbalanced datasets. The weighted random sampler applies a weight for each data entry, influencing the probability of selecting the sample in the current batch. Data points with a higher weight are selected with a higher probability therefore increasing the likelihood of sampling from the minority class. This decreases the class imbalance due to a higher likelihood of data samples belonging to the "no blocking" label, and therefore the model learns from an equal amount of positive and negative class samples. Due to the nature of randomly selecting samples based on applied weights, all samples inside a batch are randomized. The sampler is only applied during model training. When randomizing time-dependent data, such as atmospheric blocking, a strict cut between training and validation data is important to stop time leaks between the datasets. Randomization can therefore only be applied after the validation split, to make sure no days in the training dataset are included in the validation data. This issue is further discussed in Section 6.2.

```
1 train_counts = torch.bincount(labels)
2 train_class_weights = len(labels) / (2.0 * train_counts.float())
3 train_weights = train_class_weights[labels]
4 train_sampler = WeightedRandomSampler(train_weights, len(labels))
```

Listing 4.7: Setup for random weighted data sampler. The weights are paired with each data sample with the inverse class frequency across the dataset. This ensures batches with evenly distributed positive and negative samples.

Data Augmentation Pipeline

With the small ERA5 and UKESM datasets, overfitting is a major issue in model training. Propulate is used to evaluate an image augmentation pipeline on model performance. Data augmentations are provided by *alumentations* [40], a fast and powerful open-source image augmentation library. Alumentations offers a wide range of image transforms including rotations, scaling, flipping, color modifications, and advanced options such as random erasing or channel dropout. Transformations are easy to implement, chain, and deploy to model training [40]. For this thesis augmentation options to search include heavy, light, and no image augmentation. The rotation transform is configured to fill border spaces, rotated in from outside the frame with zeroes. An example application of light and heavy augmentation can be seen in Figure 4.3.

- Heavy Augmentation
 - Random Channel Dropout 1-2 Channels (10% Probability)
 - Zero out 1-2 channels of the input data randomly
 - Random Rotation up to 30° (20% Probability)
 - Gaussian Blur with Kernel Size 3 to 7 (20% Probability)
 - Apply blur to avoid overfitting on features
- Light Augmentation
 - Random Channel Dropout 1 Channel (10% Probability)
 - Zero out up to 1 channel of the input data
 - Random Rotation up to 15° (10% Probability)
- No Augmentation

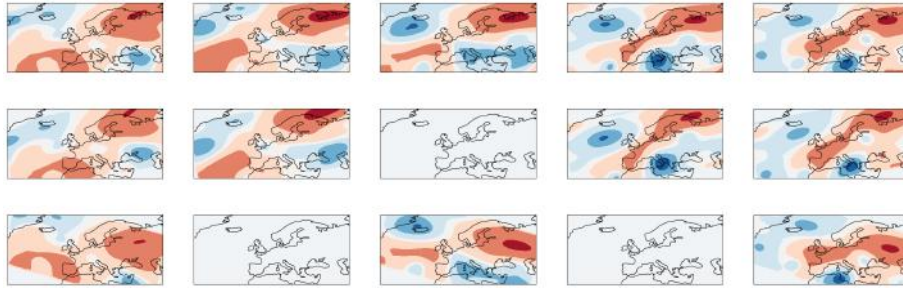


Figure 4.3: Image augmentations used during Propulate hyperparameter search visualized on the sample representing the onset of the European heat wave in 2003. The first row visualizes the unmodified data, second row is using light augmentations with channel 2 dropped and counterclockwise rotation of 10° . The last row visualizes heavy data augmentation with channels 1 and 3 dropped, a Gaussian blur applied, and clockwise rotation of 18° . This is an extreme example, with all transformations applied at the same time. In normal use, transformations are applied rarely.

Batch Size

Batch size is the number of samples evaluated for each training step before backpropagation. Larger batch size can improve generalization as more samples are included in each backpropagation run. With a smaller batch size models can have a slower convergence but capture smaller details in the input data. This thesis sets a range for batch size including small batches of 8 up to large batches of 512 samples.

Optimizer

Optimizers are crucial for model performance as they guide the model toward a lower loss value for each training step. Some optimizers also have hyperparameters that alter the behavior significantly. Training with a suboptimal optimizer can lead to slow convergence and underwhelming performance. To achieve good generalization a range of optimizers need to be evaluated to solve the exploration vs. exploitation issue for the training and validation dataset in use. This thesis evaluates a range of three optimizers commonly used in machine learning including stochastic gradient descent (sgd), adam optimizer (adam), and adagrad (adagrad). Two versions of stochastic gradient descent are available: one with momentum and one without. Momentum can aid convergence by maintaining momentum between updates, resulting in faster convergence to a minimum.

Dropout Propability

Dropout is a regularization technique widely used to combat overfitting and increase model generalization. Dropout layers randomly zero out input neurons forcing the model to learn robust representations instead of highly specific neurons. High dropout probabilities can lead to slower training times as the model adjusts to generate more robust patterns [41]. Lower dropout reduces training time but increases model likelihood for overfitting. A large dropout range from 0.0 to 0.9 is used in the hyperparameter search.

L2 Weight Decay

L2 weight decay, also called L2 penalty is a different technique to combat overfitting in machine learning. Weight decay is used inside the optimizer and punishes large weights inside the model. By punishing large weights, the model is encouraged to utilize smaller weights, increasing generalization and reducing overfitting to single examples in the training dataset. A high weight decay increases training time and needs to be balanced to ensure model convergence. Weight decay can also result in the model not being able to generalize

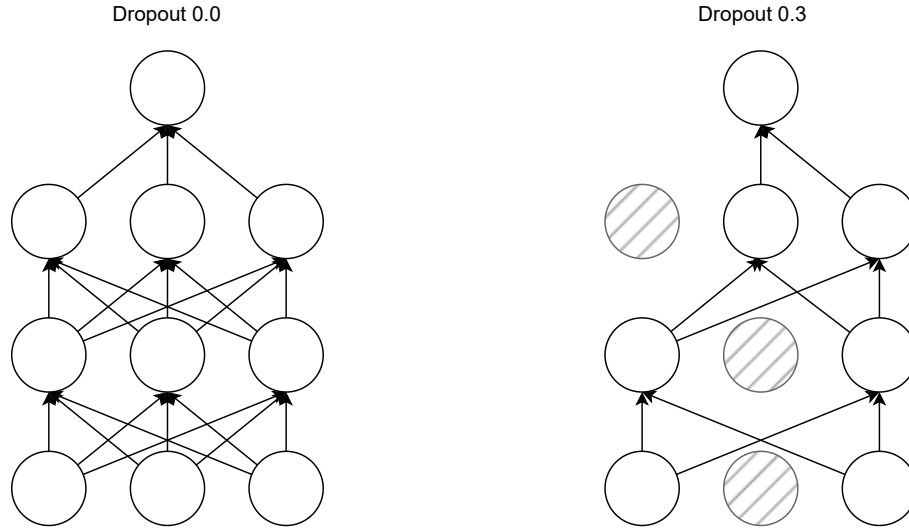


Figure 4.4: Dropout in a simple feed-forward neural network with two hidden layers and a final output neuron. In the network without dropout (left) with fully connected layers and with a dropout of 0.3 (right) some neurons are dropped.

to learn patterns inside the data, also called underfitting. As this thesis utilizes small datasets with a high risk of overfitting a large range for weight decay is searched with Propulate, ranging from 0.0 to 1.5.

```

1 limits = {
2     "model": ("resnet18", "resnet50", "efficientnet_s", "efficientnet_m", "
3     inception"),
4     "scheduler": ("step_01", "step_09", "plateau", "none"),
5     "loss": ("bce", "bce_weighted"),
6     "sampler": ("weighted_random", "none"),
7     "augmentation": ("light", "heavy", "none"),
8     "lr": (0.01, 0.0001),
9     "batch_size": (8, 512),
10    "optimizer": ("sgd_0", "sgd_09", "adam", "adagrad"),
11    "dropout": (0.0, 0.9),
12    "weight_decay": (0.0, 1.5),
13 }

```

Listing 4.8: Parameter grid used during Propulate hyperparameter search for CNNs. String tuples are searched by entry, for floats and ints the range excluding boundary numbers is searched.

Testing Pipeline

For the final CNN test, two different pipelines are set up. Both use the hyperparameter configuration found during the initial hyperparameter search. For training and testing on the same dataset, a cross-validation pipeline is set up according to Section X. For transferability testing, the model is trained and tested on different datasets. The hyperparameters used are available in the Appendix. For runs involving model pre-fitting and fine-tuning on different datasets, the stored model weights are loaded using PyTorch's `torch.load_state_dict()` function and fine-tuned according to section 4.3.

5. Results

After training and validating all hyperparameters, the final configuration for all methods is found and used to run the final training and testing runs. For each run, the used hyperparameters can be found in the appendix, clearly labeling the configuration used. The following sections present the findings for both novel supervised detection methods based on random forest classifiers and convolutional neural networks, called RF-BI and CONV-BI according to the classifier used. The results are structured into multiple paragraphs, with a focus on important training configurations and applications of RF-BI and CONV-BI. During the evaluation of the presented results, the research questions stated in Section 1.2 are answered by presenting training results and corresponding implications. In general, the resulting models from this thesis show good performance and learning capabilities for the detection of summer blocking events across Europe. Strong learning behavior during training can be observed with the models learning to classify dataset-independent blocking patterns and features, transferring accuracy between observational and model data. The performance of RF-BI, the simple supervised learning approach method already outperforms previous detection methods, with CONV-BI, a more complicated and specialized model, further improving scores.

5.1 F1 Performance

The main comparison between traditional index-based detection methods and machine-learning-based methods, such as SOM-BI, RF-BI, and CONV-BI, is based on F1 values provided by Thomas et al. [4]. For traditional blocking methods, the index is applied to the data, and results are recorded. For learning-based methods, training and testing are performed on the same dataset to use learning data similar to the final testing data. SOM-BI training and testing methodology is beyond this thesis but can be found in detail in the corresponding paper [4]. For RF-BI and CONV-BI, training and testing methodology explained in Section 4 is used. For this two meteorological variables, geopotential height and mean sea level pressure, are used to compare performance between methods, with geopotential height performing better across the board due to the higher noise level in mean sea level pressure data (Section 2.1). Additionally, a direct comparison between geopotential height and mean sea level pressure for the classification of a blocking event in 1980 is presented, with geopotential height significantly outperforming mean sea level pressure. The following research question is answered when presenting the F1 performance results in the next sections:

RQ1: How does the performance of the supervised learning approach using a convolutional neural network (CNN) compare to existing unsupervised methods and index-based methods, in detecting atmospheric blocking events during European summer?

Comparing F1 performance among the novel supervised learning models, index-based and unsupervised learning methods, a clear trend is visible. Both RF-BI and CONV-BI,

consistently outperform traditional index-based methods by a large margin. In addition, supervised methods consistently perform better than unsupervised methods. This performance increase is especially large for geopotential height data and for both observational and model data.

Dataset	Method	F1	Precision	Recall	Days blocked
ERA5	GTD	1	1	1	33.4%
ERA5	SOM-BI	0.74	0.73	0.75	34.6%
ERA5	AGP	0.56	0.76	0.44	19.5%
ERA5	DG83	0.73	0.72	0.75	34.3%
ERA5	BLO	0.50	0.33	1	100.0%
ERA5	RF-BI	0.77	0.81	0.73	33.9%
ERA5	CONV-BI	0.79	0.76	0.81	34.7%
UKESM	GTD	1	1	1	29.0%
UKESM	SOM-BI	0.71	0.70	0.72	29.6%
UKESM	AGP	0.34	0.41	0.29	20.8%
UKESM	DG83	0.60	0.90	0.45	14.5%
UKESM	BLO	0.45	0.29	1	100.0%
UKESM	RF-BI	0.80	0.84	0.77	28.8%
UKESM	CONV-BI	0.77	0.68	0.88	29.4%

Table 5.1: Comparison table adapted from Thomas et al. [4]. Performance of blocking indices AGP and DG83 compared to machine learning methods including previous SOM-BI method and novel supervised methods RF-BI and CONV-BI. All scores are relative to the ground truth dataset (GTD). Comparison for both ERA5 and UKESM datasets using geopotential height Z_{500} . For RF-BI and CONV-BI the best performer is chosen and the results across the entire dataset with the previously defined testing pipeline. Each score highlights the best performer for ERA5 and UKESM data.

Geopotential Height

On geopotential height data (Table 5.1), machine learning approaches outperform traditional blocking indices by a large margin, with supervised methods outperforming unsupervised methods across the board. For both, observational and model data, the novel supervised methods outperform traditional blocking indices and unsupervised methods consistently.

Observational Data

On ERA5 observational data (introduced in Section 3.1), CONV-BI achieved the highest performance with an F1 score of 0.78, closely followed by RF-BI with 0.77 (Table 5.1). Next up, the unsupervised learning-based method SOM-BI and traditional blocking index DG83 achieved an F1 score of 0.74 and 0.73 respectively. Both supervised methods lead the F1 score for geopotential height, with CONV-BI improving upon the previously best performer, SOM-BI, by 0.04.

The difference between RF-BI and CONV-BI is marginal with F1 differing by 0.02. Comparing both supervised methods, RF-BI performs notably better in precision, achieving a 0.05 higher F1 score compared to CONV-BI, classifying fewer false negatives. In contrast to precision, CONV-BI outperforms RF-BI in recall by 0.08. This difference in precision and recall indicates that RF-BI classifies 'blocking' with high confidence and focuses on true positives, with few false positives. At the same time, CONV-BI is better at detecting a broader range of 'blocking' events but therefore includes more false positives in its classification.

The highest-performing supervised method, CONV-BI, outperforms the blocking index DG83, with a difference of 0.05 in the F1 score. For the AGP index, the difference increases to 0.22 and is especially large, highlighting the index’s poor performance. This big difference is related to a bad recall performance and AGP not detecting most of the blocking events in the data. Adding a trivial blocking detection method, that is labeling every data sample as blocked (BLO), an F1 score of 0.50 is achieved, useful for baseline comparisons. Notably, this baseline performance almost matches the performance of AGP with a difference of 0.06. Compared to RF-BI, CONV-BI, SOM-BI, and DG83 the performance increase is clearly visible compared to a trivial baseline.

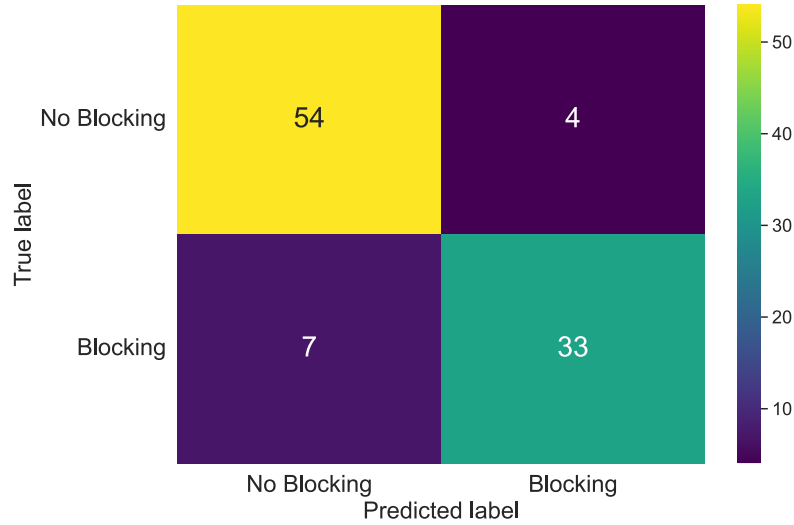


Figure 5.1: Confusion matrix generated during training and testing CONV-BI on observational (ERA5) data. This specific confusion matrix corresponds to testing in the year 1982 and training in the other 40 years. In total the year 1982 has 98 data samples with 40 blocking and 59 non-blocking labels. The model successfully predicts 33 blocking and 54 non-blocking labels. This prediction results in an F1 score of 0.86 for the year 1982.

Model Data

On UKESM model data (introduced in Section 3.1), both novel supervised methods outperform other methods with F1 scores of 0.80 for RF-BI and 0.77 for CONV-BI (Table 5.1). For model data, the discrepancy between supervised methods and others is larger than F1 performance on observational. This is clear when comparing supervised methods, especially RF-BI to traditional index-based methods and SOM-BI. RF-BI outperforms AGP by 0.46 and DG83 by 0.20, demonstrating massively increased performance. In addition, RF-BI outperforms SOM-BI by 0.09, almost doubling the performance increase on observational data.

All methods except supervised RF-BI and CONV-BI lose substantial performance between observational data and model data. The highest loss is achieved by blocking index DG83, performing well on observational data with an F1 score of 0.73, losing 0.17 on model data, and achieving a score of 0.60. SOM-BI has the smallest performance drop except for RF-BI and CONV-BI, only losing 0.03 compared to observational data.

This massive difference in model data performance clearly shows supervised methods learning to detect blocking patterns and features in the data successfully without relying on specific observational data features and thresholds. SOM-BI performs well on UKESM data compared to traditional blocking indices but loses performance compared to observational data. RF-BI and CONV-BI can match observational performance with RF-BI increasing F1

score by 0.03. The trivial detection method BLO achieves an F1 score of 0.45, outperforming AGP and decreasing the difference between DG83 to 0.15 (compared to a difference of 0.23 on observational data), further highlighting the loss in performance for index-based methods on model data.

Mean Sea Level Pressure

All methods achieve lower performance on mean sea level pressure compared to geopotential height. This is probably due to higher noise levels, explained further in Section 2.1. Resulting in hard-to-detect blocking events with noise patterns and features. When running on the mean sea level pressure data, both novel methods perform comparable or slightly better than SOM-BI. Good performance in mean sea level pressure is important for applications that are not possible with different variables including geopotential height, further examined in Section 6.4. Looking at the results, unsupervised and supervised methods perform similarly with only small differences. Actual results are in Table 5.2 with precision and recall values missing for SOM-BI. All values are measured according to the labels provided by the GTD.

Dataset	Method	F1	Precision	Recall
ERA5	GTD	1	1	1
ERA5	SOM-BI	0.64	-	-
ERA5	RF-BI	0.66	0.74	0.59
ERA5	CONV-BI	0.67	0.74	0.60
UKESM	GTD	1	1	1
UKESM	SOM-BI	0.64	-	-
UKESM	RF-BI	0.62	0.73	0.54
UKESM	CONV-BI	0.61	0.68	0.56

Table 5.2: F1 performance of SOM-BI, RF-BI, and CONV-BI on mean sea level pressure (MSLP) data. All methods are applied to ERA5 and UKESM datasets respectively. For SOM-BI precision and recall values are missing for both datasets. Traditional blocking indices are not evaluated for *MSLP*. For F1 score, the highest performer is highlighted.

Observational data results show a small edge for RF-BI and CONV-BI compared to SOM-BI, with an F1 score of 0.67 for CONV-BI and 0.66 for RF-BI. Improving by 0.03 compared to SOM-BI’s F1 score of 0.64. For supervised methods, the low score can be attributed to a low recall of 0.59 for RF-BI and 0.60 for CONV-BI. Both supervised methods perform notably higher in precision, achieving 0.74, 0.14 higher than recall. A model with relatively high precision and lower recall is successfully labeling blocking events with a high success rate but misses a lot of blocking events in the input data. This is due to the higher noise level in *MSLP* data, leading to blocking events with noisy features (Section 2.1).

On model data, the results shift slightly in favor of SOM-BI, outperforming both novel supervised methods by a small margin (0.02) with an F1 score of 0.64. Again, for RF-BI and CONV-BI a big difference between precision and recall can be observed, achieving 0.54 and 0.56 respectively. This indicates the same issue for both, observational and model data. The detection methods are unable to detect a large amount of blocking events, probably due to increased noise levels compared to geopotential height.

Compared to geopotential height, the performance deficit for *MSLP* data is very high. When comparing the highest performer for geopotential height on ERA5 (CONV-BI with an F1 score of 0.79) to mean sea level pressure (CONV-BI with an F1 score of 0.67) the drop in F1 score is 0.12. The drop in performance is apparent in a direct comparison of performance for geopotential height and mean sea level pressure data in Figure 5.2. This

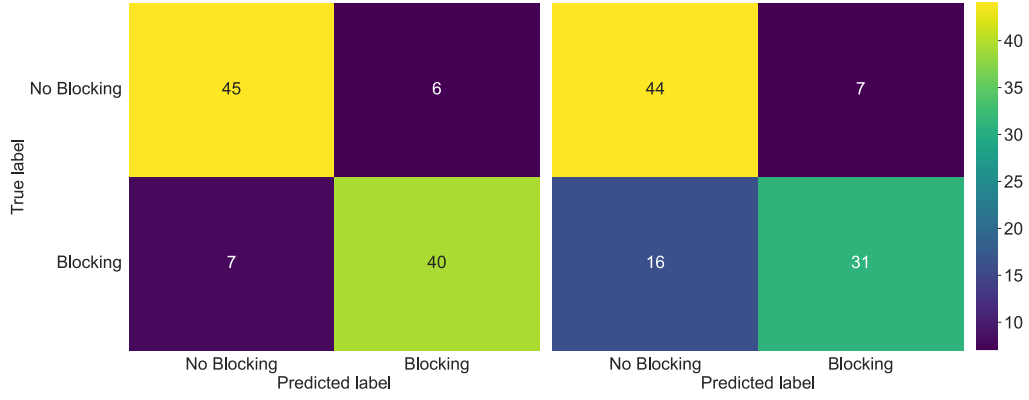


Figure 5.2: Confusion matrix comparison for CONV-BI on geopotential height (left) and mean sea level pressure (right) data with predicted labels on the x-axis and true labels on the y-axis. Both confusion matrices are generated from ERA5 training runs with the testing dataset containing the year 2003. CONV-BI on geopotential height achieved a higher performance with 7 false negatives and 6 false positives while CONV-BI on mean sea level pressure predicted 16 false negatives and 7 false positives. The F1 score for 2003 on geopotential height data is 0.88 and for mean sea level pressure data 0.77. The year 2003 is a good performing year for CONV-BI. With a difference of 0.11 for the F1 score and 10 more prediction misses, the higher performance on geopotential height data is apparent. For mean sea level pressure, the low recall performance (Section 5.1) is visible. Compared to geopotential height, CONV-BI fails to classify 9 additional blocking events.

level of performance on mean sea level pressure is lower than traditional blocking indices and unsupervised approaches. Only the worst-performing BI, AGP, achieves a lower F1 score. Even with noticeably lower F1 scores compared to geopotential height, there are applications for supervised detection methods on *MSLP*. RF-BI and CONV-BI can match the previous highest performance for blocking detection on MSLP while providing multiple advantages to previous detection methods (Section 6.1), further explained in Section 6.1.

The first research question can be concluded based on the good results obtained from the supervised methods on geopotential height and mean sea level pressure data. These methods outperform existing unsupervised and index-based methods in both variables and across observational and model data. As for the second research question, a conclusion can also be drawn from the results of the extensive hyperparameter search using Propulate.

RQ2: How do different configurations of the CNN model, such as varying architectural designs and hyperparameter choices, impact the accuracy and efficiency of blocking detection?

During training, a wide range of configurations including neural network architecture, random forest hyperparameters, and convolutional neural network hyperparameters are searched, validated, and tested. The impact on accuracy and efficiency varies for each parameter. Among these parameters, neural architecture has the greatest impact on convolutional neural networks, shifting F1 performance by up to 10%, and significantly influencing model efficiency. For large CNNs, such as Inception, the duration of training increased up to **5x** compared to smaller models such as ResNet18, and more efficient models such as EfficientNetS. Regularization hyperparameters also impact performance greatly including data augmentation levels, dropout, and weight decay. For the random forest implementation, the hyperparameters vary performance by **2-5%**. A deep dive into the parameters included in the Propulate search is documented in Section 4.3. A thorough result of the hyperparameter optimization for both CNN and RF is presented in the appendix. This includes all combinations used during training and testing to gather

the results presented in this thesis.

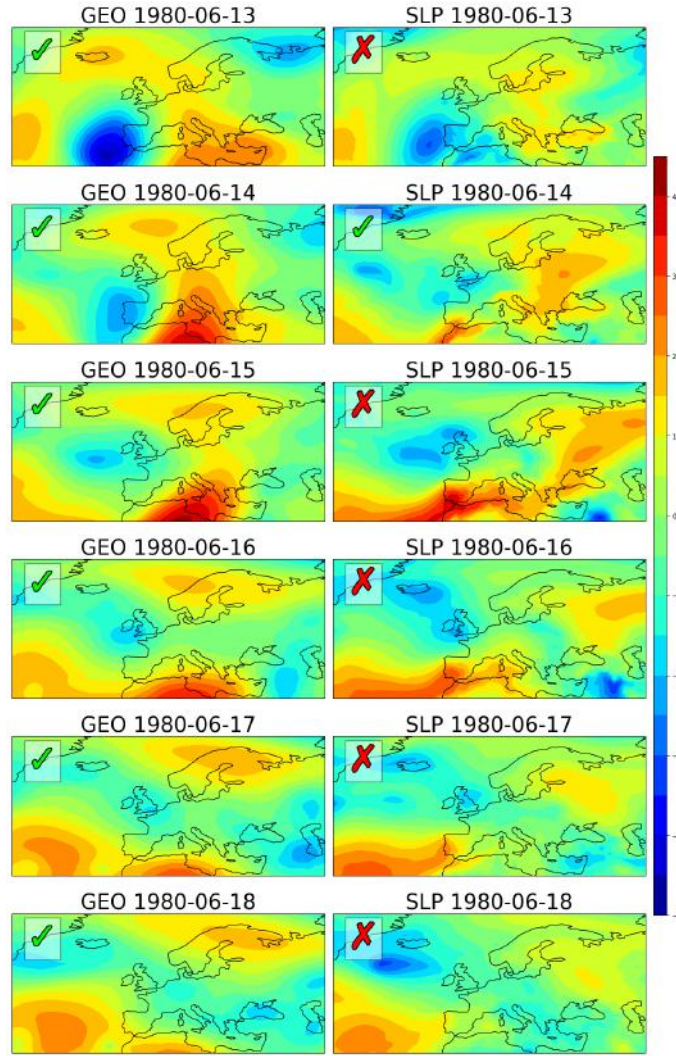


Figure 5.3: Direct comparison between ERA5 geopotential height (left) and mean sea level pressure (right) data. Classifications are taken from RF-BI within the full 41-fold training and testing setup and displayed in the top left corner for each day. The six plots for each variable range from 13.06.1980 to 18.06.1980 and represent a six-day blocking event over Europe. In the GTD all days are labeled as a positive blocking event, therefore RF-BI should detect a blocking event on geopotential height and mean sea level pressure data as well. This example highlights the difference between geopotential height and mean sea level pressure, with the blocking event over the Nordics visible in all days for geopotential height and only for some in mean sea level pressure. RF-BI struggles to detect blocking in mean sea level pressure data, only correctly detecting the blocking event for 14.06.1980. When working with geopotential height data, RF-BI is able to detect all six days as blocking, perfectly matching the ground truth dataset.

5.2 Transferability

In addition to accuracy, an important characteristic of blocking indices is the transferability of performance between different datasets. Applications for blocking detection methods require good performance on observational reanalysis data (Section 3.1) or earth system model data (Section 3.1) and therefore detection methods need to perform well on both datasets. For both novel supervised detection methods, a transferability test, similar to Thomas et al. [4], is conducted. Classifiers are trained on observational data (ERA5), tested on model data (UKESM), and vice versa. By training and testing on different datasets, the

consistency of RF-BI and CONV-BI is tested. To perform well between datasets, CONV-BI and RF-BI need to learn robust blocking patterns inside the input data to detect blocking successfully on multiple datasets. Additionally to transferability between datasets, an experiment to transfer learnings across geographical regions is conducted. Both RF-BI and CONV-BI, are used to classify a long-lasting blocking event during 2004 in the Gulf of Alaska, giving insights into geographical transferability.

For blocking detection in multiple earth system models, a good performance on model data while training on observational data, is a good indicator. This hints at a good performance on earth system models without prior knowledge of the model data. Similarly, if the model performs well on observational data after training with simulated model data, it demonstrates the potential to use data from models for training instead of relying only on actual measurements from Earth’s atmosphere. This greatly increases the potential for dataset size and therefore model performance. The results will also answer parts of the third research question in the next paragraphs.

RQ3: How does the CNN model perform on simulated data when trained on observational data or a combination of observational and simulated data, and what implications does this have for the model’s applicability in real-time or near-real-time atmospheric blocking prediction?

To evaluate the transferability of model performance, similar to Thomas et al. [4], each model is trained on one dataset and tested on a different kind. The meteorological variable used during the transferability tests is geopotential height. Mean sea level pressure transferability is not tested during this thesis.

Train Dataset	Test Dataset	Method	F1	Precision	Recall
-	ERA5	GTD	1	1	1
UKESM	ERA5	SOM-BI	0.74	-	-
UKESM	ERA5	RF-BI	0.76	0.83	0.70
UKESM	ERA5	CONV-BI	0.72	0.76	0.68
-	UKESM	GTD	1	1	1
ERA5	UKESM	SOM-BI	0.71	-	-
ERA5	UKESM	RF-BI	0.77	0.78	0.76
ERA5	UKESM	CONV-BI	0.73	0.71	0.77

Table 5.3: Transferability of RF-BI and CONV-BI between observational ERA5 data and UKESM model data. All models are trained on one dataset and tested on a different dataset. Performance is compared to the ground truth dataset. The top rows represent training on the UKESM dataset and testing on ERA5. The bottom rows contain results for training on ERA5 and testing on UKESM. For SOM-BI, values are provided by Thomas et al. [4] and miss precision and recall performance.

When testing on model data (UKESM), both supervised methods demonstrate good performance, only losing marginally compared to training and testing on model data directly. RF-BI achieves the highest F1 score for this comparison with 0.77, outperforming CONV-BI with 0.73 and SOM-BI with 0.71. Compared to the performance during training and testing on model data (Section 5.1), RF-BI and CONV-BI lose 0.03 and 0.04 respectively while SOM-BI remains constant with an F1 score of 0.71 for both runs. The small loss in performance for both supervised methods could hint at slight overfitting to observational data or could be the result of small differences between observational and model data. Even with a small loss in performance, both supervised methods outperform SOM-BI, with RF-BI increasing the F1 score by 0.06.

For observational data (ERA5), the results are similar. RF-BI and CONV-BI demonstrate

good performance with F1 scores of 0.76 and 0.72 respectively. SOM-BI also performs well with an F1 score of 0.74, slightly higher than CONV-BI but lower compared to RF-BI. Comparing the performance to training and testing on observational data (Section 5.1), RF-BI loses 0.01 and CONV-BI 0.07, hinting again at slight overfitting and data discrepancies between observational and model data.

Both tests demonstrate that RF-BI and CONV-BI are effective in learning robust blocking patterns that can be transferred across multiple datasets. Although there is a slight decrease in F1 performance compared to training and testing directly on the same dataset, strong learning on the test dataset can be observed. Both RF-BI and CONV-BI outperform SOM-BI in most combinations. These results are promising for applications that require transferability of blocking detection between observational and model data.

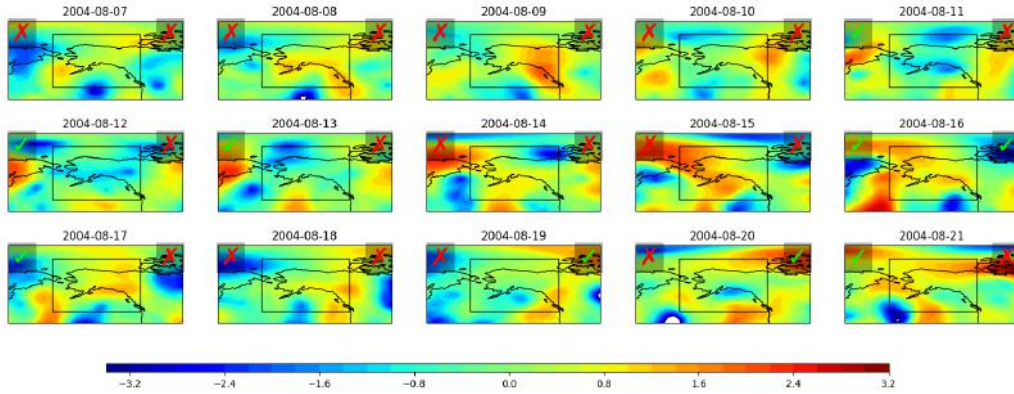


Figure 5.4: Regional transferability experiment during a large atmospheric blocking event in the Gulf of Alaska in 2004. Hussain et al. [42] published a detailed analysis of this event, suggesting the blocking onset period from 02.08 to 05.08 and the total event lasting until 28.08.2004. For this experiment, RF-BI and CONV-BI, are used to classify this long-lasting atmospheric blocking event. To do this, the corresponding ERA5 geopotential height dataset is downloaded and preprocessed according to Section 3.3. Importantly, the input area is cut to the same dimensions used with European summer blocking data and re-gridded equally. RF-BI and CONV-BI are trained on ERA5 European summer geopotential height data and then used to classify 02.08.2004 to 28.08.2004. In this figure, 15 days in the middle of the blocking event are visualized with the black rectangle in the middle above the blocking event in the Gulf of Alaska described by Hussain et al. [42]. CONV-BI predictions are indicated for each day in the top left corner, while RF-BI predictions are in the top right. A checkmark represents a blocking classification and a cross vice versa. CONV-BI classifies 6 out of 15 days as blocking while RF-BI only classifies 3. With the suggested 100% blocking rate in this data excerpt suggested by Hussain et al. [42], both methods perform badly, with CONV-BI detecting 40% and RF-BI 20%. Compared to European summer data, this is a sharp decline in performance for both detection methods. CONV-BI manages to outperform RF-BI by a significant margin, hinting at better transferability and generalization for CONV-BI. This result could be because of multiple influences, such as data preparations and differences in blocking definition between Hussain et al. [42] and the labels in the GTD by Thomas et al. [4]. After preprocessing the data, there is no clear high-pressure blocking event visible on multiple days during August.

5.3 Combined Training & Model Fine-Tuning

Supervised learning algorithms need large amounts of data to achieve high performance (Section 2.2) and therefore struggle with the small datasets used during this thesis. This issue is alleviated partly by utilizing transfer learning from already pre-trained convolutional neural networks (Section 4.3), heavy regularization, and data augmentation (Section 4.3) but can not be fully resolved without expanding the available training data. With the high

transferability results between both available datasets, a training setup with combined observational and model data is tested. This increases the dataset size to 142 years consisting of 13916 daily samples. Compared to previous observational and model data this represents an increase of 346% and 140% in dataset size respectively.

The following results will answer the previously stated question about model performance on a combination of observational and model data.

RQ3: How does the CNN model perform on simulated data when trained on observational data or a combination of observational and simulated data, and what implications does this have for the model’s applicability in real-time or near-real-time atmospheric blocking prediction?

Combined Training

For combined training, both datasets are merged to create a new dataset with observational and model data including the corresponding labels. For this dataset, one run without shuffling the training data and one run with shuffled data is performed. The results indicate no performance difference for shuffling, so all reported results are recorded from runs without shuffling. For final testing, models are trained using a different k -fold setup to the previously introduced and utilized setup, folding only the testing dataset during training. This results in 41 folds for observational (ERA5) data and 101 folds for model (UKESM) data explained in Figure 5.5.

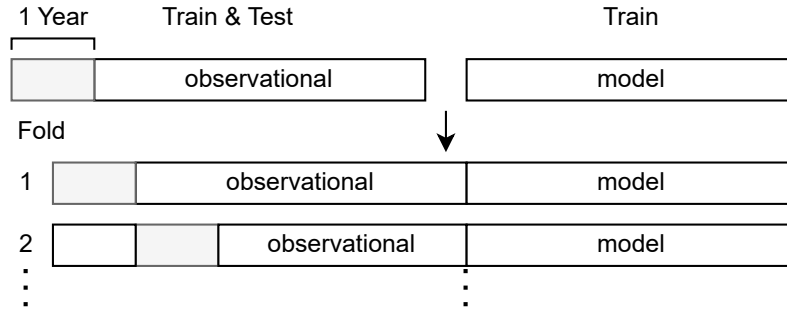


Figure 5.5: Final train and test setup for combined observational and model datasets. In this visualization, observational data is used for testing. Therefore, the yearly folds are only present on observational data (left). Model data (right), which is only used during training, is added to the training data after each fold. The final number of folds is equal to the number of years in the testing dataset.

Fine-Tuning

A different method commonly used for CNNs called fine-tuning is also tested for CONV-BI. Fine-tuning utilizes one of the datasets to pre-train the model and only applies final adjustments with the second dataset. This approach is similar to the already utilized pre-trained models and utilizes either observational or model data during pretraining before fine-tuning on the other dataset. This decreases computational load, as the pre-trained model can be re-used for all following folds. Final training and testing is implemented according to the previously defined convolutional neural network pipeline (Section 4.3).

When testing on ERA5 data, both RF-BI and CONV-BI achieve similar performance. For CONV-BI, combined training and fine-tuning achieve an F1 score of 0.79 with RF-BI achieving a marginally lower performance of 0.78. Compared to training and testing solely on ERA5 data (Section 5.1), CONV-BI performs equally. RF-BI improves slightly from previous F1 score 0.77. When comparing to transferability scores (Section 5.2), both

Training	Fine-Tune	Test	Method	F1
-	-	ERA5	GTD	1
UKESM	ERA5	ERA5	CONV-BI	0.79
ERA5 + UKESM	-	ERA5	CONV-BI	0.79
ERA5 + UKESM	-	ERA5	RF-BI	0.78
-	-	UKESM	GTD	1
ERA5	UKESM	UKESM	CONV-BI	0.80
ERA5 + UKESM	-	UKESM	CONV-BI	0.81
ERA5 + UKESM	-	UKESM	RF-BI	0.81

Table 5.4: Performance for RF-BI and CONV-BI with fine-tuning a pre-trained model and training on combined observational and climate model data.

supervised methods improve notably in performance. CONV-BI and RF-BI increase F1 score by 0.07 and 0.02 respectively. This is expected, as both methods can train on a subset of the dataset type used during testing. Interestingly, RF-BI is not benefitting similarly to CONV-BI when comparing the transferability scores. This could be due to RF-BI already performing better than CONV-BI during transferability and therefore not learning any additional information on the combination of ERA5 and UKESM data.

The results of both supervised methods are similar when testing on model data. CONV-BI and RF-BI perform well, with no noticeable difference between CNN fine-tuning and training on both datasets simultaneously. CONV-BI achieves an F1 score of 0.81, improving accuracy directly on ERA5 by 0.04. RF-BI performance shows no significant improvement, with an F1 score of 0.81 in both instances. Compared to the transferability of supervised methods (Section 5.2), CONV-BI benefits significantly from the larger dataset, improving F1 score by 0.08. RF-BI improves performance by 0.04 still showing a noticeable difference.

The presented results for both observational and model data show small improvements compared to direct training (Section 5.1). For CONV-BI a larger improvement can be noticed compared to RF-BI's smaller differences. This is unexpected, as more training data should help to prevent overfitting and increase model performance. The combination of model and observation data could confuse both methods due to different data sources and therefore not improve testing results significantly. Further investigation is necessary and is discussed later during the conclusion (Section 6.4).

RQ3: How does the CNN model perform on simulated data when trained on observational data or a combination of observational and simulated data, and what implications does this have for the model's applicability in real-time or near-real-time atmospheric blocking prediction?

To conclude on the third research question, a combination of observational and model data during training does not increase performance significantly. When looking at transferability scores, both supervised methods show strong improvements, but this is most likely due to data from the same dataset in testing also contained in the training data.

6. Discussion and Conclusion

The following chapter discusses the presented results and draws a conclusion, including a brief look at the strengths and weaknesses of the novel detection methods RF-BI and CONV-BI. The chapter introduces learning and tricks, such as data augmentation, heavy regularization, and data shuffling, used during training and fine-tuning of both methods. Furthermore, the limitations of this work and the factors influencing the obtained results are presented. A brief outlook on possible future work with a focus on advances in supervised blocking detection is given at the end.

6.1 Method Comparison

This thesis presents two novel blocking detection methods based on supervised learning algorithms. RF-BI and CONV-BI demonstrate strong performance, consistently outperforming other traditional index-based and unsupervised learning methods. In addition to raw performance, both algorithms demonstrate significant strengths and advantages over other detection methods. After presenting the strengths, the following sections also include a look at the weaknesses and drawbacks of both RF-BI and CONV-BI.

Strengths

Both RF-BI and CONV-BI show **good transferability scores** between ERA5 and UKESM datasets (Section 5.2), implying a good general understanding of blocking patterns, independent of the data used during training. Transferability is one of the key benefits of machine learning for blocking detection, enabling models to reliably detect blocking in a wide range of data sources. For example, models trained on a mixture of observational and model data can be used to detect blocking in an ensemble of earth system models used for long-term trend analysis of climate change scenarios or short-term blocking detection in simulations. Consistent performance across multiple datasets enables for better comparison of detection methods and could imply transferability between different geographic regions, which is discussed further in Section 6.4.

With consistent performance across datasets, models can **scale by utilizing multiple datasets** during training to increase dataset size. Models can be fitted on larger, simulated, or combined datasets and fine-tuned on specific observational or earth system model data to enhance performance additionally. General blocking patterns are learned on a larger dataset and smaller, dataset-specific, details are fine-tuned. To increase performance even further, basic image patterns and filters can be re-used from large CNN models trained on ImageNet [16] further explained in Section 4.3. By increasing the size of labeled blocking datasets, the accuracy of supervised detection methods such as RF-BI and CONV-BI increases further.

Supervised blocking detection methods are **easy to set up and use during inference** (Figure 6.1). RF-BI is stored using the `pickle` module available in Python and can be

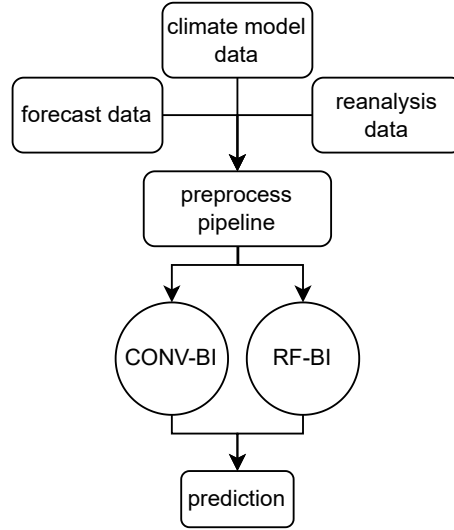


Figure 6.1: Inference setup for RF-BI and CONV-BI blocking detection methods. Multiple data sources are preprocessed and inputted into the models to produce a final prediction.

loaded using `pickle.load()`. The sklearn model can be used directly after unpickling or deployed to a server or cloud platform such as Amazon SageMaker. CONV-BI weights are saved by pickling the model `state_dict` with `torch.save(model)` and can be loaded by using `torch.load_state_dict()` and supplying the weights to the original model. The restored PyTorch model can then be used for inference or deployed using Tensorserve, Amazon SageMaker, or a custom server.

```

1 model = Model(*args)
2 model.load_state_dict(torch.load(path_to_weights))
3 model.eval()

```

Listing 6.1: Python code to load state dict containing model weights from disk and initializing a trained model. The model is set into eval mode and ready for inference.

Weaknesses

Both algorithms train on the same **expert labeled dataset** provided by Thomas et al. [4] consisting of 41 years of observational and 101 years of earth system model data. To increase model performance by providing more training data, expensive expert labeling is required.

Another issue with the training data is the **specific geographic region**. With the training data only including blocking events during European summer, RF-BI and CONV-BI are only trained on European summer blocking events. There is no test dataset to evaluate performance on different geographic regions and blocking events. The high transferability of results between observational and model data implies a general understanding of blocking patterns and features by both methods but is not enough to conclude on performance outside of the tested geographic region and season. A small experiment is conducted to test transferability for a long-lasting blocking event in the Gulf of Alaska in 2004 (Section 5.2), but no big comparisons including different labeled datasets.

The final model prediction is only binary to indicate the presence or absence of a blocking event in the input data. In contrast to traditional blocking indices, RF-BI and CONV-BI, produce no output map with blocking detections on each grid cell. Therefore, both supervised algorithms are unable to pinpoint the location of a detected blocking event

or indicate the blocked region directly in maps. Applications relying on the location of blocking cells inside data samples can not use RF-BI and CONV-BI in their current form. A different approach of using CNNs to output a 2D blocking detection map is possible and further discussed in Section 6.4.

Due to the small datasets available for blocking detection, **overfitting** is a major problem during the training of both RF-BI and CONV-BI. Especially CONV-BI tends to overfit on both observational and model data, with UKESM performing better due to $2.4\times$ more data samples than ERA5. Multiple methods to combat overfitting are already implemented in the training pipeline for RF-BI and CONV-BI (Section 4.3, such as data augmentation, dropout, and weight decay. The need for large, labeled datasets remains one of the largest weaknesses of supervised learning methods, and heavy regularization with data augmentation only helps to some degree. In the end, only collecting more data will substantially increase final model performance.

The inference setup for supervised methods is simpler but a preprocessing pipeline for RF-BI and CONV-BI is still necessary. To achieve accurate predictions, the data has to be processed with the same pipeline used during training and validation to ensure correct de-seasonalization, de-trending, and normalization. This includes all steps used during preprocessing and the final data dimensions used during training. The preprocessing pipeline is discussed in-depth in Section 3.3.

6.2 Lessons Learned

Learnings and Tricks

During the training of both methods, certain tricks and learning techniques significantly improved the final result. These are particularly interesting for future work building on supervised learning in blocking detection, especially for CNNs. In the following sections, learnings and tricks are presented with a short explanation and the corresponding impact on the performance of the methods.

Data Augmentation

The scarcity of labeled data for blocking detection is one of the main issues in training larger networks. Especially deep neural networks, such as CNNs, suffer from smaller datasets with a high tendency to overfit. In machine learning, so-called *data augmentation* is often used to artificially inflate small datasets. With data augmentation certain operations are applied to existing data samples, keeping the original data while adding additional augmented data samples to reduce model overfitting. Algorithms used for image augmentation include geometric transformations, channel augmentations, filters, blurs, noise, and feature erasing [43]. For this thesis, an image augmentation pipeline with different augmentation levels is implemented. The final image augmentation pipeline increases F1 scores by **1-2%** across the board. An example application of the image augmentations in use is presented in Figure 4.3.

Heavy Regularization

Traditional regularization approaches are used in conjunction with image augmentation to further increase the generalization of trained methods. Dropout [41] is one of the most widely spread regularization techniques, zeroing out neurons during training to force the underlying model to generalize and learn robust patterns in the input data [14]. A different, widely spread regularization technique is L2 parameter regularization also called weight decay. By punishing large parameters during backpropagation, weight decay ensures that no neurons specialize themselves to specific input samples and therefore dominate the output. This forces the model to generalize and not focus on single features in the input data [14]. L2 regularization together with dropout increase model performance by **1-2%**.

Data Shuffling

All supervised learning algorithms are trained and fitted using a series of five days as input data. Blocking is a time-dependent phenomenon defined with a length of five days up to multiple weeks. By training on samples containing five days the model can classify blocking states correctly but this introduces overlap with specific days included in up to five data entries at the same time. For robust final results, it is crucial to separate training, validation, and test data in yearly intervals to avoid data leaking between the splits. If validation and test data are seen by the model during training the final results are skewed due to the model learning on days included in validation and testing. To avoid this indirect leaking of validation and testing data into the training dataset it is important to avoid shuffling the data before splitting it into subsets for training, validation, and testing. Certain frameworks and functions shuffle data samples by default, therefore taking a closer look into utilized helper functions is important. Widely used functions shuffling the underlying data distribution include `sklearn.model_selection.train_test_split`, `sklearn.model_selection.KFold`, samplers from `torch.utils.data` and `torch.utils.data.DataLoader`. Shuffling data before training, validation, and test split wrongfully increases F1 performance by **6-8%**.

Factors Influencing the Results

In addition to learning and techniques, certain factors can influence the results without being noticeable during training and testing. These factors include dataset-specific characteristics and time dependence between data samples. The following sections present the factors that can influence the results in addition to the configuration of the supervised algorithm. For future work in supervised blocking detection, it is important to take these factors into account.

Yearly Overlapping

Due to the date range cut to JJA (June-July-August) for summer blocking events, the data is not circular. Therefore, at the end of each year, the last days are connected to the first days of the following year to form five-day data samples needed for classification according to Section 3.3. This introduces data samples with 9 months between days therefore eliminating the possibility to represent real blocking events. In this case, the corresponding data points are labeled negative. An example for this jump in the data is presented in Figure 6.2.

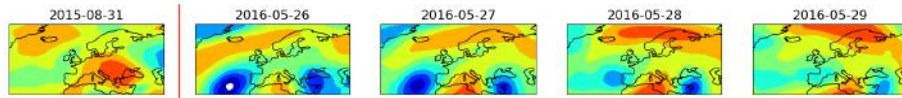


Figure 6.2: Overlap between 2015 (first) and the beginning of 2016 (second to fifth) with a nine-month gap and no connection between geopotential height state for the first and following days.

Data Availability

As the final results are calculated corresponding to the GTD and all supervised methods are trained and fitted with labels provided by the GTD this paper relies heavily on the expert labeling done by Thomas et al. [4]. For all human-labeled datasets, a small error rate is inevitable and impacts performance claims during training, validation, and testing. This error is also present for all other comparison methods, such as traditional blocking indices and unsupervised learning, and should therefore not skew comparison scores between methods too much.

Furthermore, the labeled dataset is limited to the European region during the summer months (June-July-August) and therefore the methods are trained and fitted on European summer blocking events. It is important to note that SOM-BI, RF-BI, and CONV-BI are specialized to the European region and not tested for application in different regions. For CONV-BI and RF-BI an experiment to classify a long-lasting blocking event in the Gulf of Alaska during 2004 is conducted 5.2. Additionally, they are only trained on summer blocking events and no events during winter. The good transferability between observational data and climate model data indicates a good generalization to detect blocking patterns not specific to the training data used.

With a hard-defined domain area bounding box, the model is defined to the geographic region over Europe, extending to the Atlantic Ocean and Russia. This is important for model training and fitting and is predefined by the GTD [4]. Due to the dynamic nature of blocking it could be helpful for certain events to have an input with more global features that could hint at the formation of a blocking event. A larger receptive field could help RF-BI and CONV-BI to pick up outside information connected to blocking in the European geographic area.

Due to the rarity of blocking events, the dataset is inherently imbalanced. Blocking is an atmospheric phenomenon and therefore it is impossible to have a good balance between positive and negative data samples. For machine learning algorithms utilizing backpropagation, this imposes the issue of imbalanced class distributions. The model is presented more samples from the majority class and the weights are adjusted to correctly classify the samples seen during training. This thesis tries to combat class balance by utilizing weighted upsampling of the minority class and weighted binary cross entropy loss to increase the impact of samples from the minority class explained further in Section 4.3.

Longitude and Latitude Box

To create the GTD Thomas et al. excluded blocking events on the edge of the domain area even when a large region inside the domain could be considered blocked [4]. In the case of supervised learning, algorithms work with the entire domain data for blocking events and can therefore detect large blocked areas at the edge of the data with a corresponding negative label. There is a trade-off about which border events to include as border events can rely on important features outside of the domain area to be classified correctly.

Performance Metrics

The performance metric used for comparisons and method optimization also influence the results. This is especially true with the F1 score, a relation between precision and recall. Visible in Table 5.1 the F1 score is dependent on a high precision and recall score simultaneously, punishing methods with a large gap between them. In Table 5.1 AGP scored the lowest F1 score for traditional blocking indices with 0.56 while performing well in precision (0.76) but badly in recall (0.44). Therefore AGP might be a good blocking detection method for applications that rely on a few mistakes in classifying non-blocking samples as blocking (few false positives), as AGP is performing well in precision. The F1 score is a standard evaluation metric in a wide range of machine learning tasks and is therefore a good baseline to compare certain methods, but the underlying relation between precision and recall and the implications behind the F1 score should be understood when choosing an algorithm based on F1 performance.

6.3 Study Limitations

The following section introduces certain limitations in the study regarding computational, data, and time limits. These limitations are interesting for future research topics and advances in supervised learning in blocking detection.

HAICORE

For computational resources, the high computing cluster HAICORE introduced in Section 3.4 is used. HAICORE enables the use of state-of-the-art hardware geared towards machine learning for large neural networks. HAICORE limits access for users to 4 A100 GPUs simultaneously and a maximum job time of 8:00 hours, which enables training of neural networks and small parameter searches but is insufficient for large training cycles and hyperparameter searches with long-running jobs. Therefore the following training situations were limited by HAICORE and not fully evaluated:

- Propulate Hyperparameter Search Setup
 - Limited to one search per dataset and variable
 - No separate searches for each test run (each test year)
- Propulate Hyperparameter Search ERA5
 - Limited to a population size of 8 individuals (2 per A100 GPU)
 - Limited to 4 generations (repeated 5 times)
- Propulate Hyperparameter Search UKESM
 - Limited to a population size of 8 individuals (2 per A100 GPU)
 - Limited to 3 generations (repeated 5 times)
- Convolutional Neural Network Architecture
 - Largest model with Inception architecture (Section 4.3)
- Batch size
 - Limited to 512 to fit 2 individuals in a single A100 GPU with 40GB RAM

With a higher availability of compute resources, a larger and more thorough Propulate hyperparameter search can be utilized to increase performance further.

Data Availability

One of the main limiting factors is the availability of labeled training data. In climate research, this is often one of the major issues in utilizing supervised machine learning approaches and has previously been solved by publishing large, community-sourced datasets such as ClimateNet [20]. For blocking detection the only available data is published by Thomas et al. [4] and includes 41 years of observational and 101 years of climate model data. After preprocessing both datasets combined a total of 13.916 data samples. For deep learning methods such as CNNs, this is a small dataset and is further split into observational and climate model data. With 4.018 samples for observational data, this is a major limiting factor for model training and final performance. Using pre-trained CNNs and regularization techniques such as image augmentation and weight decay, this thesis artificially enlarges the dataset and forces models to learn general patterns, but can not fully alleviate the lack of sufficient training data.

Due to a labeled dataset available only for the European region and the summer months (June-July-August), this thesis is limited to the geographic region and time frame of the available training data. With no available labels for different regions, the transferability between different geographical regions is not evaluated, except for a small experiment with a long-lasting blocking event during 2004 in the Gulf of Alaska (Section 5.2).

Train, Validation and Test Split

Due to the small dataset size, this thesis utilizes a different training, validation, and testing data split that does not strictly divide the different steps. Instead of the usual up-front train, validation, and test splits, such as 60% training, 20% validation, and 20% testing data, this thesis uses k -fold cross-validation on the entire input data. This introduces intentional data leakage during hyperparameter search and cross-validation between training and testing, by choosing hyperparameters with the final testing data included. During an experiment with the data split into test, training, and validation, before choosing hyperparameters, performance was similar. Therefore, this thesis values higher test dataset size and robust final results over an up-front testing split. Nonetheless, it is important to note the data leakage due to hyperparameters not being searched for each testing run separately.

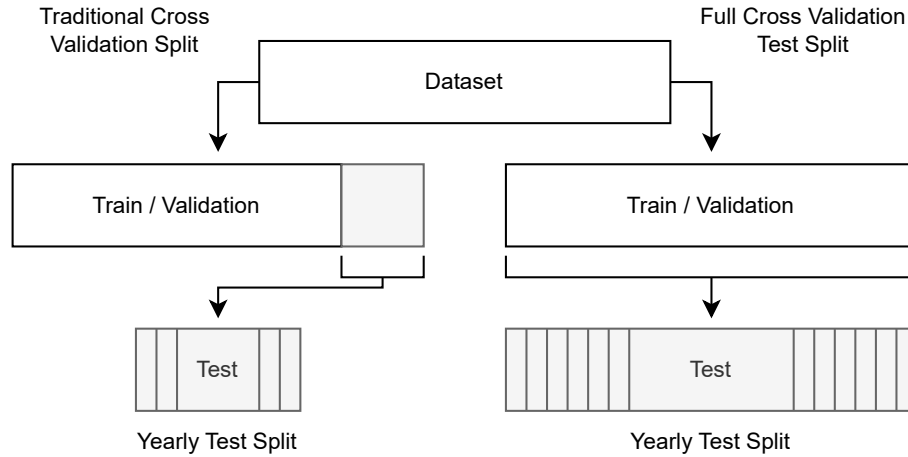


Figure 6.3: Comparison of a normal train, validation, and test setup (left) to the setup used in this thesis (right). The split used in this thesis utilizes the entire dataset for cross-validation and parameter search and does not divide a separate test set first. The final yearly test setup is equal for both approaches.

6.4 Conclusion and Outlook

The following section will conclude on the findings of this thesis and the potential for supervised learning in future applications. A general outlook on possible future developments is also discussed.

Summary of Key Findings

This bachelor thesis introduces two new novel methods for blocking detection using a supervised machine learning approach, called RF-BI and CONV-BI.

Both methods demonstrate good performance across the board when trained and tested on observational data (ERA5) and earth system model data (UKESM). This includes performance on geopotential height (Z_{500}) and mean sea level pressure ($MSLP$) data. For observational data, provided by the European Centre for Medium-Range Weather Forecasts, both methods demonstrate increased performance compared to traditional blocking indices and unsupervised methods on geopotential height. In a comparison of 101 years of pre-industrial climate model data, provided by the UK Earth System Modelling Project, both supervised learning models, RF-BI and CONV-BI demonstrate strong learning behavior outperforming traditional blocking indices and unsupervised methods with a significant margin. For geopotential height, especially on model data, supervised learning demonstrates great improvements compared to all other methods.

Performance on mean sea level pressure (*MSLP*), a more robust meteorological variable compared to geopotential height, is important when applications require a stable variable for long-term trend analysis. Due to higher noise in *MSLP* data, the performance for all methods is noticeably lower compared to Z_{500} . For observational data, RF-BI and CONV-BI perform marginally better than SOM-BI. Performance on climate model data demonstrates a similar performance for RF-BI, CONV-BI, and SOM-BI. Both introduced supervised methods, RF-BI and CONV-BI are suitable for mean sea level pressure analysis.

In addition to performance on mean sea level pressure data, the performance of both supervised learning methods in a transfer between observational and model data is tested. Both methods show a strong learning transfer, demonstrating the ability to learn abstract and general blocking patterns and perform well when used in a cross-dataset setup. This includes good performance when training on observational and testing on model data and vice versa. Both supervised methods lose marginally compared to training and testing on the same dataset but still demonstrate strong prediction behavior, outperforming SOM-BI in all instances. Compared to SOM-BI, performance on observational data when training on simulated model data is up noticeably. For transferring learnings from observational data to model data, the improvements of RF-BI and CONV-BI are smaller.

Utilizing both datasets to train a single model using fine-tuning or a combined dataset does not increase performance notably for supervised methods RF-BI and CONV-BI. Only CONV-BI improves consistently with more available data, hinting at larger improvements with larger datasets from the same source or model. For CONV-BI, fine-tuning of a pre-fitted model on one of the datasets demonstrated similar performance to training on both datasets combined. This decreased computational load by 40-60% while maintaining high performance.

By introducing RF-BI and CONV-BI, this thesis presents results and outlooks into machine learning enhancing our ability to predict and understand atmospheric blocking. These methods not only outperform traditional and unsupervised approaches but also demonstrate versatility across different data types and scenarios. The promising results from transferring learning between observational and model data suggest that machine learning models can adapt to various data sources, offering a more flexible and robust tool for meteorological research. While the improvements from combined training were limited, the ability to learn on larger datasets from the same source could improve performance significantly. Overall, the findings underscore the significant role that supervised learning plays in advancing atmospheric science, particularly in the precise detection of blocking events.

Outlook on Possible Future Developments

The thesis presents the potential of supervised blocking detection, leaving room for future developments and research.

Increasing the size of the dataset and including more data sources is of particular interest, as it has the potential to greatly improve performance, significantly outperforming traditional detection methods. This could be in the form of more labeled datasets in multiple geographic regions, or additional labeled Earth system model data. The addition of more data increases performance for all supervised methods, while making comparisons and results more robust.

An interesting addition to supervised blocking methods is the dimensionality of the output. Novel supervised methods presented in this paper produce a binary output on the presence of a blocking event in the input data. This is useful for certain applications, but often an output map corresponding to a blocking prediction for each grid cell is required. This is more accurate and can be used to geographically locate the blocked area. For convolutional neural networks, image-to-image models are commonly used for image segmentation and

use several architectures, including encoder-decoder and attention-based models [44]. This approach can also be used to generate an output map (single-channel image) of block detections corresponding to specific grid cells in the input data. This process requires two-dimensional labels for blocking events, which can be expertly labeled or taken from existing blocking indices.

Future research can investigate the transferability of detection capabilities for blocking events in multiple geographic regions. Additional observational data from blocking events with similar features and patterns from different geographical regions can be used for training if model performance translates well between both regions. During application, a single model can classify multiple regions known to contain blocking events during summer or winter. A general input pipeline can be established to preprocess multiple regions in a similar way to apply a single model to all of them.

Other supervised algorithms, such as Visual Transformers or Long Short-Term Memory models, can also be tested for detecting atmospheric blocking. Additionally, blocking data can be used with models that take advantage of the timely dependence between multiple days, increasing the information used for classification instead of ignoring the time aspect. Larger CNNs also have the potential to further increase performance. In addition to supervised detection methods, another interesting area of research is in the form of self-supervised detection methods. Models utilizing self-supervised learning can learn details of blocking by reconstructing existing events or predicting future events. This reduces the dependency on labeled data and helps models already learn basic blocking features that can be fine-tuned with smaller labeled datasets.

In conclusion, this thesis has shown how supervised methods can detect atmospheric blocking and highlighted approaches to improve the newly introduced methods in the future. Adding more and varied data could greatly improve the performance of these methods and outperform traditional ways of detecting blocking significantly. In addition, changing method output to 2D maps that show regional blocking detections could provide more detailed information, similar to traditional blocking indices. In the future, testing other supervised algorithms and incorporating self-supervised learning could provide new ways to better understand and predict atmospheric blocking with higher accuracy. This work is a starting point for deeper research into supervised blocking detection.

Appendix

Code Availability

All code used during this paper can be found in a public GitHub repository: <https://github.com/tobiasleibrock/blocking>. Files in the `bin/` directory are used for data preprocessing and visualizations with Jupyter Notebook. The `model/` directory contains all files used during RF-BI and CONV-BI training and testing. This includes all data-related functionality in `dataset.py` and utility functions in `util.py`. All random forest functionality is stored in a Jupyter Notebook (`rf.ipynb`). Convolutional neural network hyperparameter search, training, and testing functionality are available in `propulate_search.py`, `model.py` and `train.py`.

For dependency management, the repository utilizes Poetry (<https://python-poetry.org/>). Any other dependency manager can also be used with the `pyproject.toml`.

Additional setup instructions related to running the code and reproducing the results can be found in the project README.

Data Availability

The Ground Truth Dataset (GTD) used for labeling ERA5 and UKESM data is provided by Thomas et al. [4]. Instructions for download and use are available in the paper. Additional ERA5 and UKESM datasets can be downloaded via the official European Centre for Medium-Range Weather Forecasts [5] and The UK Earth System Modelling project [25] channels. No additional datasets were used during model training and evaluation.

Code Listings

```
1 def deseasonalization(dataset):
2     daily_climatology = dataset.groupby("time.dayofyear").mean(dim="time")
3     window_size = 10
4
5     daily_climatology_circular = xr.concat(
6         [
7             daily_climatology.isel(dayofyear=slice(-window_size//2, None)),
8             daily_climatology,
9             daily_climatology.isel(dayofyear=slice(None, window_size // 2))
10        ],
11        dim="dayofyear",
12    )
13
14    daily_climatology_circular = daily_climatology_circular.rolling(
15        dayofyear=10, center=True, min_periods=1
16    ).mean()
17
18    daily_climatology = daily_climatology_circular.isel(
```

```

19     dayofyear=slice(window_size // 2, -window_size // 2)
20 )
21
22     ds_deseasonalized = dataset - daily_climatology.sel(
23         dayofyear=dataset.time.dt.dayofyear
24     )
25     return ds_deseasonalized

```

Listing 1: Python function used for de-seasonalization of all input datasets. A yearly rolling mean is calculated for each grid cell and subsequently subtracted from every data point in the input. Dependencies for this function include Xarray and numpy.

Visualizations, Graphics, and Tables

Data	Arch	Scheduler	Optim	Loss	Sampler	Aug	LR	Batch	Drop	Penalty
ERA5-GEO	Inception	Step01	Adagrad	Weighted	None	Heavy	4.19E-3	157	5.06E-3	4.70E-1
ERA5-MSLP	EfficientM	Plateau	SGD0	Weighted	None	Heavy	7.33E-3	83	2.12E-1	2.06E-1
UKESM-GEO	EfficientM	Step01	SGD0	BCE	None	Light	6.67E-3	121	2.18E-1	2.26E-1
UKESM-MSLP	Inception	Plateau	Adam	BCE	None	Light	7.03E-3	145	4.38E-2	3.13E-1
ERA5-GEO	Resnet50	Step01	SGD9	Weighted	None	Light	5.75E-3	159	3.67E-1	1.38E-1
ERA5-MSLP	Resnet50	Step09	Adagrad	BCE	None	Heavy	2.84E-3	211	1.92E-1	2.93E-2
UKESM-GEO	Inception	Step09	Adagrad	Weighted	None	Light	3.52E-3	105	2.23E-1	3.98E-1
UKESM-MSLP	Resnet50	Step09	Adagrad	BCE	None	Light	2.84E-3	211	1.92E-1	2.93E-2

Table .1: Propulate hyperparameter search results for CNN training on ERA5 and UKESM. The results are split into GEO (geopotential height) and MSLP (mean sea level pressure) data. The top rows represent the highest-performing configuration found using Propualte. The bottom rows represent the runner-up configuration. Column labels represent hyperparameters, with "Arch" referring to pre-trained model architecture, "Optim" referring to the used optimizer, "Aug" referring to the level of data augmentation used, "Drop" referring to the dropout probability, and "Penalty" referring to the weight decay penalty used. The search limits are explained in-depth in Section 4.3. "Step" refers to a step scheduler with a factor added to the name. For example, "Step01" is a step scheduler with a factor of 0.1. The same naming is used for "SGD", stochastic gradient descent. For example "SGD9" refers to a stochastic gradient descent loss function with momentum 0.9.

Data	Max Features	Min Leaf	Min Split	Estimators
ERA5-GEO	log2	4	5	400
ERA5-MSL	log2	1	5	400
UKESM-GEO	sqrt	1	10	400
UKESM-MSL	sqrt	4	10	200
ERA5+UKESM	log2	1	10	200
ERA5-GEO	sqrt	2	2	200
ERA5-MSL	log2	4	5	400
UKESM-GEO	log2	4	5	400
UKESM-MSL	sqrt	1	2	200
ERA5+UKESM	log2	2	10	400

Table .2: Hyperparameter optimization results using grid search with a Random Forest classifier on ERA5 and UKESM data. The table is organized into GEO (geopotential height) and MSLP (mean sea level pressure) datasets. The top section shows the best-performing configurations, while the bottom section presents the runner-up configurations. Column labels indicate the hyperparameters evaluated: "Max Features" for the number of features to consider when looking for the best split, "Min Leaf" for the minimum number of samples required to be at a leaf node, "Min Split" for the minimum number of samples required to split an internal node, and "Estimators" for the number of trees in the forest.

Bibliography

1. Fouillet, A., Rey, G., *et al.* Has the impact of heat waves on mortality changed in France since the European heat wave of summer 2003? A study of the 2006 heat wave. *International Journal of Epidemiology* **37**, 309–317. doi:10.1093/ije/dym253 (2008-01).
2. R. García-Herrera J. Díaz, R. M. T. *et al.* A Review of the European Summer Heat Wave of 2003. *Critical Reviews in Environmental Science and Technology* **40**, 267–306. doi:10.1080/10643380802238137 (2010).
3. Nakamura, N. & Huang, C. S. Y. Atmospheric blocking as a traffic jam in the jet stream. *Science* **361**, 42–47. doi:10.1126/science.aat0721 (2018).
4. Thomas, C., Voulgarakis, A., Lim, G., *et al.* An unsupervised learning approach to identifying blocking events: the case of European summer. *Weather and Climate Dynamics* **2**, 581–608. doi:10.5194/wcd-2-581-2021 (2021).
5. Hersbach, H., Bell, B., *et al.* ERA5 hourly data on single levels from 1940 to present. *Copernicus Climate Change Service (C3S) Climate Data Store (CDS)*. Accessed on 07.01.2024. doi:10.24381/cds.adbb2d47 (2023).
6. Woollings, T., Barriopedro, D., *et al.* Blocking and its response to climate change. *Current climate change reports* **4**, 287–300 (2018).
7. Kautz, L.-A., Martius, O., Pfahl, S., *et al.* Atmospheric blocking and weather extremes over the Euro-Atlantic sector – a review. *Weather and Climate Dynamics* **3**, 305–336. doi:10.5194/wcd-3-305-2022 (2022).
8. Rex, D. F. Blocking Action in the Middle Troposphere and its Effect upon Regional Climate. *Tellus* **2**, 196–211. doi:https://doi.org/10.1111/j.2153-3490.1950.tb00331.x (1950).
9. Sardeshmukh, P. D. & Hoskins, B. J. The Generation of Global Rotational Flow by Steady Idealized Tropical Divergence. *Journal of Atmospheric Sciences* **45**, 1228–1251. doi:10.1175/1520-0469(1988)045<1228:TGOGRF>2.0.CO;2 (1988).
10. Davini, P., von Hardenberg, J. & Corti, S. Tropical origin for the impacts of the Atlantic Multidecadal Variability on the Euro-Atlantic climate. *Environmental Research Letters* **10**, 094010. doi:10.1088/1748-9326/10/9/094010 (2015-09).
11. Rosenblatt, F. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review* **65**, 386–408. doi:10.1037/h0042519. https://doi.org/10.1037/h0042519 (1958).
12. Fradkov, A. L. Early History of Machine Learning. *IFAC-PapersOnLine* **53**. 21st IFAC World Congress, 1385–1390. doi:https://doi.org/10.1016/j.ifacol.2020.12.1888 (2020).
13. Wang, H. & Raj, B. *On the Origin of Deep Learning* 2017. arXiv: 1702.07800 [cs.LG].

14. Goodfellow, I., Bengio, Y. & Courville, A. *Deep Learning* <http://www.deeplearningbook.org> (MIT Press, 2016).
15. Halevy, A., Norvig, P. & Pereira, F. The Unreasonable Effectiveness of Data. *IEEE Intelligent Systems* **24**, 8–12. doi:10.1109/MIS.2009.36 (2009).
16. Deng, J., Dong, W., Socher, R., *et al.* *ImageNet: A large-scale hierarchical image database* in *2009 IEEE Conference on Computer Vision and Pattern Recognition* (2009), 248–255. doi:10.1109/CVPR.2009.5206848.
17. James, G., Witten, D., Hastie, T., *et al.* *An Introduction to Statistical Learning: with Applications in Python* (Springer, 2013).
18. De Burgh-Day, C. O. & Leeuwenburg, T. Machine learning for numerical weather and climate modelling: a review. *Geoscientific Model Development* **16**, 6433–6477. doi:10.5194/gmd-16-6433-2023 (2023).
19. Rolnick, D. *et al.* Tackling Climate Change with Machine Learning. *ACM Comput. Surv.* **55**. doi:10.1145/3485128 (2022-02).
20. Prabhat, Kashinath, K., *et al.* ClimateNet: an expert-labeled open dataset and deep learning architecture for enabling high-precision analyses of extreme weather. *Geoscientific Model Development* **14**, 107–124. doi:10.5194/gmd-14-107-2021 (2021).
21. Chattopadhyay, A., Hassanzadeh, P. & Pasha, S. Predicting clustered weather patterns: A test case for applications of convolutional neural networks to spatio-temporal climate data. *Scientific Reports* **10**, 1317. ISSN: 2045-2322. doi:10.1038/s41598-020-57897-9 (2020).
22. Dee, D. P., Balmaseda, M., *et al.* Toward a Consistent Reanalysis of the Climate System. *Bulletin of the American Meteorological Society* **95**, 1235–1248. doi:10.1175/BAMS-D-13-00043.1 (2014).
23. Sellar, A. A. *et al.* UKESM1: Description and Evaluation of the U.K. Earth System Model. *Journal of Advances in Modeling Earth Systems* **11**, 4513–4558. doi:<https://doi.org/10.1029/2019MS001739> (2019).
24. UK Earth System Model. *UKESM - UK Earth System Model* <https://ukesm.ac.uk/>. Accessed on 25th February 2024. 2024.
25. Tang, Y., Rumbold, S., *et al.* *MOHC UKESM1.0-LL model output prepared for CMIP6 CMIP historical* 2019. doi:10.22033/ESGF/CMIP6.6113.
26. NSF Unidata, Davis, G., *et al.* *NetCDF* doi:10.5065/D6H70CW6.
27. Schulzweida, U. *CDO User Guide* version 2.3.0. 2023-10. doi:10.5281/zenodo.10020800.
28. Hoyer, S. & Hamman, J. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software* **5**. doi:10.5334/jors.148 (2017).
29. Paszke, A., Gross, S., *et al.* *PyTorch: An Imperative Style, High-Performance Deep Learning Library* 2019. doi:10.48550/arXiv.1912.01703. eprint: 1912.01703.
30. Pedregosa, F., Varoquaux, G., Gramfort, A., *et al.* Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011).
31. High Performance and Cloud Computing Group. *HAICORE Documentation* Accessed on 2 March 2024. <https://www.nhr.kit.edu/userdocs/haicore/>.
32. Taubert, O., Weiel, M., Coquelin, D., *et al.* *Massively Parallel Genetic Optimization Through Asynchronous Propagation of Populations in High Performance Computing* (Springer Nature Switzerland, 2023), 106–124. ISBN: 978-3-031-32041-5. doi:10.1007/978-3-031-32041-5_6.

33. Propulate Documentation. *Propulate Documentation* Accessed on 29 February 2024. <https://propulate.readthedocs.io/en/latest/>.
34. Raschka, S. *Model Evaluation, Model Selection, and Algorithm Selection in Machine Learning* 2020. arXiv: 1811.12808 [cs.LG].
35. Zhuang, F., Qi, Z., Duan, K., *et al.* A Comprehensive Survey on Transfer Learning. *Proceedings of the IEEE* **109**, 43–76. doi:10.1109/JPROC.2020.3004555 (2021).
36. He, K., Zhang, X., Ren, S., *et al.* *Deep Residual Learning for Image Recognition* 2015. arXiv: 1512.03385 [cs.CV].
37. Tan, M. & Le, Q. V. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* 2020. arXiv: 1905.11946 [cs.LG].
38. Tan, M. & Le, Q. V. *EfficientNetV2: Smaller Models and Faster Training* 2021. arXiv: 2104.00298 [cs.CV].
39. Szegedy, C., Vanhoucke, V., Ioffe, S., *et al.* *Rethinking the Inception Architecture for Computer Vision* 2015. arXiv: 1512.00567 [cs.CV].
40. Buslaev, A., Iglovikov, *et al.* Albumentations: Fast and Flexible Image Augmentations. *Information* **11**, 125. ISSN: 2078-2489. doi:10.3390/info11020125 (2020-02).
41. Srivastava, N., Hinton, G., Krizhevsky, A., *et al.* Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research* **15**, 1929–1958 (2014).
42. Hussain, A. & Lupo, A. Scale Analysis of Blocking Events from 2002 to 2004: A Case Study of an Unusually Persistent Blocking Event Leading to a Heat Wave in the Gulf of Alaska during August 2004. *Advances in Meteorology* **2010**. doi:10.1155/2010/610263 (2010-01).
43. Shorten, C. & Khoshgoftaar, T. M. A survey on Image Data Augmentation for Deep Learning. *Journal of Big Data* **6**, 60. ISSN: 2196-1115. doi:10.1186/s40537-019-0197-0 (2019).
44. Minaee, S., Boykov, Y., *et al.* *Image Segmentation Using Deep Learning: A Survey* 2020. eprint: 2001.05566 (cs.CV).

Acknowledgement

This work is supported by the Helmholtz Association Initiative and Networking Fund on the HAICORE@KIT partition.

We acknowledge the World Climate Research Programme, which, through its Working Group on Coupled Modelling, coordinated and promoted CMIP6. We thank the climate modeling groups for producing and making available their model output, the Earth System Grid Federation (ESGF) for archiving the data and providing access, and the multiple funding agencies who support CMIP6 and ESGF.

The ERA5 data used in this study/project/research was obtained from the Copernicus Climate Change Service Climate Data Store (CDS).

The results contain modified Copernicus Climate Change Service information 2020. Neither the European Commission nor ECMWF is responsible for any use that may be made of the Copernicus information or data it contains.

The UKESM1-0-LL dataset, version 3.1, was utilized for the research presented in this thesis. This dataset is part of the UK Earth System Model (UKESM) project. The development and distribution of the UKESM1-0-LL dataset by the UKESM project, under the radiative forcings of CMIP v6.2.0, are gratefully acknowledged. The author would like to thank the UKESM project for providing the necessary resources and support.